

## АНОТАЦІЯ

Бакалаврська кваліфікаційна робота виконана студентом групи ІІІ-42 Ковалем Денисом Петровичем. Тема «Система для автоматичної нормалізації суржику та діалектних форм сучасної української мови». Робота направлена на здобуття ступеня бакалавра за спеціальністю 122 «Комп'ютерні науки».

Об'єктом дослідження є процеси автоматичної нормалізації нелітературних форм сучасної української мови - суржику та регіональних діалектів.

Предметом дослідження є методи формування паралельних корпусів для низькоресурсних мовних варіантів, а також методи навчання нейронних моделей машинного перекладу та великих мовних моделей для задачі нормалізації діалектного і суржикового тексту.

У роботі зібрано паралельний корпус із 125 615 навчальних прикладів для чотирьох мовних різновидів - гуцульського, бойківського, закарпатського діалектів і суржику. Для синтетичної генерації використано комбінацію правилкових підходів і великої мовної моделі Mistral Small 24B. Навчено дві моделі: кодер-декодерну (umt5-base, повне донавчання) та декодерну велику мовну модель (MamayLM на базі Gemma-3-4B, параметро-ефективне донавчання QLoRA). Якість оцінено за метриками BLEU, chrF++ і TER у порівнянні з нуль-шотовими великими мовними моделями - GPT-4o, Gemini 2.0 Flash і Mistral Large.

Навчені моделі перевершили нуль-шотові великі мовні моделі на суржику (chrF++ = 93,58 для umt5-base) та гуцульському діалекті (chrF++ = 86,90 для umt5-base). Реалізовано вебзастосунок «Лексикон» із підтримкою голосового введення.

Загальний обсяг роботи: – сторінок, – рисунків, 18 посилань.

Ключові слова: нормалізація тексту, суржик, діалекти, нейронний машинний переклад, велика мовна модель, донавчання, параметро-ефективне налаштування, паралельний корпус, гуцульський діалект.

## ABSTRACT

The bachelor's qualification work was completed by Koval Denys Petrovych, a student of group ShI-42. The topic is "System for Automatic Normalization of Surzhyk and Dialectal Forms of the Modern Ukrainian Language". The work is aimed at obtaining a bachelor's degree in specialty 122 "Computer Science".

The object of research is the process of automatic normalization of non-standard forms of modern Ukrainian - surzhyk and regional dialects.

The subject of research is methods for building parallel corpora for low-resource language varieties, and methods for training neural machine translation models and large language models for the task of dialect and surzhyk normalization.

A parallel corpus of 125,615 training examples was collected for four language varieties: Hutsul, Boikivian, and Transcarpathian dialects, as well as surzhyk. Synthetic data was generated using a combination of rule-based approaches and the Mistral Small 24B language model. Two models were trained: an encoder-decoder model (umt5-base, full fine-tuning) and a decoder-only large language model (MamayLM based on Gemma-3-4B, QLoRA parameter-efficient fine-tuning). Quality was evaluated using BLEU, chrF++, and TER metrics against zero-shot large language model baselines: GPT-4o, Gemini 2.0 Flash, and Mistral Large.

The fine-tuned models outperformed zero-shot large language models on surzhyk (chrF++ = 93.58 for umt5-base) and the Hutsul dialect (chrF++ = 86.90 for umt5-base). A web application "Leksykon" was implemented with voice input support.

The total volume of work: – pages, – figures, 18 references.

Keywords: text normalization, surzhyk, dialects, neural machine translation, large language model, fine-tuning, parameter-efficient adaptation, parallel corpus, Hutsul dialect.

# ЗМІСТ

## Зміст

|                                                                                                                  |           |
|------------------------------------------------------------------------------------------------------------------|-----------|
| <b>АНОТАЦІЯ</b>                                                                                                  | <b>1</b>  |
| <b>ABSTRACT</b>                                                                                                  | <b>2</b>  |
| <b>ЗМІСТ</b>                                                                                                     | <b>3</b>  |
| <b>ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ</b>                                                                                 | <b>6</b>  |
| <b>ВСТУП</b>                                                                                                     | <b>7</b>  |
| <b>1 ЗАГАЛЬНІ ПОЛОЖЕННЯ</b>                                                                                      | <b>9</b>  |
| 1.1 Аналіз предметної галузі та літературних джерел . . . . .                                                    | 9         |
| 1.1.1 Нормалізація як задача між нейронним машинним пере-<br>кладом і виправленням граматичних помилок . . . . . | 9         |
| 1.1.2 Діалекти та суржик як об'єкти автоматичної нормалізації                                                    | 10        |
| 1.1.3 Наявні ресурси та підходи до синтетичного розширення<br>даних . . . . .                                    | 11        |
| 1.1.4 Сучасні україномовні моделі та найближчі суміжні роботи                                                    | 12        |
| 1.2 Аналіз вхідних даних . . . . .                                                                               | 13        |
| 1.2.1 Лінгвістична характеристика мовних різновидів . . . . .                                                    | 13        |
| 1.2.2 Ресурси для гуцульського діалекту . . . . .                                                                | 14        |
| 1.2.3 Корпус «Співавтор» як джерело літературної норми . . . .                                                   | 15        |
| 1.2.4 Словникова база мовних різновидів . . . . .                                                                | 16        |
| 1.2.5 Синтетична генерація даних . . . . .                                                                       | 17        |
| 1.2.6 Формування та структура фінального корпусу . . . . .                                                       | 18        |
| 1.3 Висновки до розділу 1 . . . . .                                                                              | 19        |
| <b>2 ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ</b>                                                                | <b>21</b> |
| 2.1 Аналіз підходів до нормалізації . . . . .                                                                    | 21        |
| 2.1.1 Нормалізація як задача перетворення тексту . . . . .                                                       | 21        |
| 2.1.2 Правильні підходи та їх обмеження . . . . .                                                                | 22        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 2.1.3    | Нейромережеві підходи: кодер-декодерна модель і декодерна ВММ . . . . . | 22        |
| 2.2      | Постановка задачі та математична модель . . . . .                       | 23        |
| 2.2.1    | Формальна постановка нормалізації . . . . .                             | 23        |
| 2.2.2    | Формат даних для двох архітектур . . . . .                              | 23        |
| 2.2.3    | Цільова функція: крос-ентропія і згладжування міток . .                 | 24        |
| 2.3      | Математичні основи трансформера . . . . .                               | 25        |
| 2.3.1    | Механізм уваги . . . . .                                                | 25        |
| 2.3.2    | Позиційне кодування, нормалізація шарів і прямозв'язний блок . . . . .  | 25        |
| 2.3.3    | Кодер-декодерна архітектура . . . . .                                   | 26        |
| 2.3.4    | Авторегресійна декодерна модель . . . . .                               | 26        |
| 2.4      | Вибір та обґрунтування моделей . . . . .                                | 27        |
| 2.4.1    | Кодер-декодерна модель umt5-base . . . . .                              | 27        |
| 2.4.2    | Декодерна ВММ MamyLM . . . . .                                          | 28        |
| 2.4.3    | Параметро-ефективне донавчання: LoRA та QLoRA . . .                     | 29        |
| 2.5      | Гіперпараметри та конфігурація навчання . . . . .                       | 29        |
| 2.5.1    | Донавчання umt5-base . . . . .                                          | 29        |
| 2.5.2    | Донавчання MamyLM (QLoRA) . . . . .                                     | 30        |
| 2.6      | Метрики оцінювання . . . . .                                            | 31        |
| 2.6.1    | BLEU . . . . .                                                          | 31        |
| 2.6.2    | chrF++ . . . . .                                                        | 32        |
| 2.6.3    | TER . . . . .                                                           | 33        |
| 2.6.4    | Взаємодоповнюваність метрик . . . . .                                   | 33        |
| 2.7      | Стратегії декодування . . . . .                                         | 34        |
| 2.8      | Балансування даних . . . . .                                            | 34        |
| 2.8.1    | Дисбаланс діалектів та принципи семплювання . . . . .                   | 34        |
| 2.8.2    | Формат інструкцій для мультидіалектності . . . . .                      | 35        |
| 2.9      | Висновки до розділу . . . . .                                           | 35        |
| <b>3</b> | <b>ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ</b>                               | <b>36</b> |
| 3.1      | Технологічний стек . . . . .                                            | 36        |
| 3.2      | Конвеєр генерації корпусу . . . . .                                     | 37        |
| 3.2.1    | Правилова генерація суржику . . . . .                                   | 37        |
| 3.2.2    | LLM-генерація для діалектів . . . . .                                   | 38        |

|                                   |                                                        |           |
|-----------------------------------|--------------------------------------------------------|-----------|
| 3.2.3                             | Підготовка корпусу до навчання . . . . .               | 40        |
| 3.3                               | Навчання моделей . . . . .                             | 41        |
| 3.3.1                             | Донавчання umt5-base . . . . .                         | 41        |
| 3.3.2                             | Донавчання MamayLM . . . . .                           | 42        |
| 3.4                               | Оцінювання та аналіз результатів . . . . .             | 44        |
| 3.4.1                             | Методологія оцінювання . . . . .                       | 44        |
| 3.4.2                             | Результати кодер-декодерної моделі umt5-base . . . . . | 44        |
| 3.4.3                             | Результати декодерної ВММ MamayLM . . . . .            | 45        |
| 3.4.4                             | Порівняння з нуль-шотовими базовими лініями . . . . .  | 45        |
| 3.4.5                             | Аналіз результатів . . . . .                           | 46        |
| 3.5                               | Вебзастосунок «Лексикон» . . . . .                     | 47        |
| 3.5.1                             | Архітектура застосунку . . . . .                       | 47        |
| 3.5.2                             | Модуль нормалізації . . . . .                          | 48        |
| 3.5.3                             | Модуль автоматичного розпізнавання мовлення . . . . .  | 49        |
| 3.5.4                             | Інтерфейс користувача . . . . .                        | 49        |
| 3.6                               | Висновки до розділу . . . . .                          | 50        |
| <b>ЗАГАЛЬНІ ВИСНОВКИ</b>          |                                                        | <b>51</b> |
| <b>СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ</b> |                                                        | <b>52</b> |
| <b>Додаток А</b>                  |                                                        | <b>54</b> |
| <b>Додаток Б</b>                  |                                                        | <b>55</b> |

## ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

АРМ - автоматичне розпізнавання мовлення (Automatic Speech Recognition) - технологія перетворення мовленнєвого сигналу на текст.

ВММ - велика мовна модель (Large Language Model) - нейронна модель на основі трансформера, навчена на великих масивах тексту та здатна до генерації і обробки природної мови.

ГП - графічний процесор - спеціалізований процесор для паралельних обчислень, що використовується при навчанні нейронних мереж.

НМП - нейронний машинний переклад (Neural Machine Translation) - підхід до автоматичного перекладу тексту з використанням нейронних мереж.

ОП - оперативна пам'ять - основна пам'ять комп'ютера для зберігання даних під час виконання програм.

ОПМ - обробка природної мови (Natural Language Processing) - галузь штучного інтелекту, що вивчає методи автоматичного аналізу і генерації текстів.

ЦП - центральний процесор - основний обчислювальний пристрій комп'ютера.

API (Application Programming Interface) - програмний інтерфейс застосунків, набір визначень і протоколів для взаємодії між програмними компонентами.

BLEU (Bilingual Evaluation Understudy) - автоматична метрика оцінювання якості машинного перекладу на основі збігу n-грамів.

chrF++ (character n-gram F-score) - метрика оцінювання якості перекладу на основі символічних і словесних n-грамів.

LoRA (Low-Rank Adaptation) - метод параметро-ефективного донавчання нейронних мереж шляхом введення низькорангових матриць адаптації.

PEFT (Parameter-Efficient Fine-Tuning) - підхід до донавчання великих нейронних мереж, при якому оновлюється лише невелика частина параметрів.

QLoRA (Quantized LoRA) - варіант методу LoRA із квантизацією ваг базової моделі до 4 біт для зменшення вимог до пам'яті.

TER (Translation Edit Rate) - метрика оцінювання якості перекладу, що вимірює кількість редагувань, необхідних для приведення результату до еталонного тексту.

## ВСТУП

**Актуальність теми.** Українська мова існує в умовах значної варіативності: поряд із літературною нормою в різних регіонах побутують гуцульський, бойківський і закарпатський діалекти, а в містах широко вживається суржик - змішане українсько-російське мовлення. Масова внутрішня міграція після початку повномасштабного вторгнення 2022 року поглибила мовні контакти: люди з різними мовними звичками опиняються в одному середовищі, де діалектні та суржикові форми можуть ускладнювати ділову комунікацію, навчання і роботу з документами. Попри очевидну потребу, спеціалізованих автоматичних інструментів нормалізації регіональних діалектів і суржику в Україні майже немає: наявні системи орієнтовані переважно на орфографічні та граматичні помилки і не розраховані на систематичне перетворення діалектних форм на літературну норму.

**Мета роботи** - розробка та дослідження системи автоматичної нормалізації суржику і трьох регіональних діалектів (гуцульського, бойківського, закарпатського) до літературного стандарту сучасної української мови на основі нейронних моделей і підходів до роботи з малоресурсними даними.

### **Завдання дослідження:**

- проаналізувати сучасні підходи до автоматичної нормалізації діалектних і гібридних мовних форм;
- обґрунтувати стратегію формування паралельного корпусу для чотирьох мовних різновидів з обмеженим обсягом ручної розмітки;
- зібрати і підготувати паралельний корпус для гуцульського, бойківського, закарпатського діалектів і суржику;
- навчити та порівняти дві моделі нормалізації - кодер-декодерну (umt5-base) і декодерну велику мовну модель (MamauLM) - на спільному багатодіалектному корпусі;
- оцінити якість навчених моделей у порівнянні з нуль-шотовими великими мовними моделями за метриками BLEU, chrF++ і TER;
- реалізувати прикладний вебзастосунок «Лексикон» із підтримкою голосового введення.

**Об'єктом дослідження** є процес автоматичної нормалізації нелітератур-

них форм сучасної української мови - суржику та регіональних діалектів.

**Предметом дослідження** є методи формування паралельних корпусів для малоресурсних мовних варіантів, а також методи навчання нейронних моделей нейронного машинного перекладу і великих мовних моделей для задачі нормалізації.

**Методи дослідження:** аналіз і формування корпусів текстових даних; синтетична генерація паралельних прикладів за допомогою лінгвістичних правил, словникових ресурсів і великої мовної моделі; навчання кодер-декодерних трансформерних моделей; параметро-ефективне донавчання великих мовних моделей (QLoRA); автоматичне оцінювання якості за метриками BLEU, chrF++ і TER.

**Наукова новизна** роботи полягає в тому, що вперше запропоновано і досліджено єдину систему нормалізації одночасно для суржику і трьох регіональних діалектів. Для бойківського та закарпатського діалектів сформовано перші публічні синтетичні паралельні корпуси. Проведено порівняльне оцінювання кодер-декодерної та декодерної архітектур на задачі мультидіалектної нормалізації з залученням нуль-шотових великих мовних моделей як базових ліній.

**Практична значущість** роботи полягає у розробці вебзастосунку «Лексикон», що нормалізує тексти гуцульського, бойківського і закарпатського діалектів та суржику до літературної української мови з підтримкою голосового введення. Систему можна використовувати як самостійний інструмент або інтегрувати до ширших конвеєрів автоматичної обробки україномовних текстів.



# 1 ЗАГАЛЬНІ ПОЛОЖЕННЯ

## 1.1 Аналіз предметної галузі та літературних джерел

### *1.1.1 Нормалізація як задача між нейронним машинним перекладом і виправленням граматичних помилок*

Автоматична нормалізація діалектних форм і суржику до літературної норми є задачею, що поєднує властивості двох напрямів обробки природної мови (ОПМ): нейронного машинного перекладу (НМП) та автоматичного виправлення граматичних помилок. Від НМП вона відрізняється тим, що перетворення відбувається між різновидами однієї мови: фонологічна, лексична та морфологічна системи діалекту частково збігаються з літературною нормою, а вхідний текст зазвичай залишається зрозумілим для носія стандарту. Від задачі виправлення граматичних помилок нормалізація відрізняється тим, що допустимі відхилення є системними - правильними в межах певного діалекту - а не випадковими помилками: регулярні фонетичні та лексичні трансформації не можна просто «виправити», не враховуючи діалектний контекст.

На практиці задача нормалізації може розв'язуватися трьома основними підходами, що еволюціонували від правилкових систем до нейронних. Правильові та словникові системи покладаються на явно задані відповідники між діалектними формами і літературними: словники заміни, регулярні перетворення морфем, фонетичні правила. Такі системи добре інтерпретовані, але погано масштабуються на нові домени і не охоплюють контекстно-залежних трансформацій. Моделі типу «послідовність-у-послідовність» на основі трансформера дозволяють навчити кодувально-декодуєчу модель на паралельних даних «діалект → норма» і природно трактують задачу як переклад усередині мови. Великі мовні моделі (ВММ) з інструкційним донавчанням поєднують генеративну потужність авторегресійних моделей і гнучкість формату запиту, що дозволяє одній моделі нормалізувати кілька різновидів за умови явної вказівки типу задачі.

Роботи з НМП для діалектів підтверджують, що багатозадачне навчання, яке об'єднує кілька мовних різновидів в одну модель, дає кращі результати за умови обмежених даних: для арабських діалектів показано, що спільна модель стабілізує навчання і узагальнює спільні лінгвістичні закономірності [1.]. Для малоресурсних сценаріїв принципово важливо, що навіть невеликий обсяг якісних паралельних даних суттєво покращує якість відносно нуля-шотового

застосування загальної моделі. Байтові підходи, як-от ВуТ5, зменшують залежність від словникових сегментацій і краще працюють із нестандартизованими текстами, що містять фонетичні відхилення [15.]. Для багатомовних сценаріїв важливою є також стратегія збалансованого вибору мов під час попереднього навчання, що впливає на якість для малоресурсних мов [3.].

### *1.1.2 Діалекти та суржик як об'єкти автоматичної нормалізації*

Чотири мовні різновиди, що досліджуються в цій роботі, суттєво відрізняються за природою і ступенем систематизованості відхилень від літературної норми.

Гуцульський діалект поширений у гірських районах Карпат - Івано-Франківській, Закарпатській та Чернівецькій областях. Він є добре задокументованим: для нього наявні лінгвістичні описи, словники та паралельні корпуси. Гуцульський вирізняється специфічними голосними змінами (наголошені «а» та «я» переходять у «є»: «як» → «єк», «шапка» → «шепка»), особливою поведінкою зворотної частки (використовується «си» замість «ся» і може стояти перед дієсловом: «він сміється» → «він си сміє»), а також характерними відмінами дієслів третьої особи однини («знає» → «знаєт»). Наявність ручних паралельних даних і словника зробила гуцульський природною відправною точкою для побудови системи.

Бойківський діалект є одним із найархаїчніших в Україні - він зберігає риси давньоукраїнської мови і праслов'янські фонологічні елементи. Характерні ознаки: розрізнення переднього [и] і заднього [ы], що зникло в стандартній мові («бик» → «бык», «тихо» → «тыхо»); перехід [л] у [љ] у кінці складу («галька» → «гавка», «кіл» → «ків»); особові енклітики в умовному способі (-бим, -бись, -бисьмо); архаїчні числівникові форми («дев'ятнадцять» → «двайцять без єнного»). Бойківський є найбільш лексично задокументованим серед чотирьох різновидів.

Закарпатський діалект вирізняється тривалим контактом із сусідніми мовами - словацькою, угорською та румунською. Збережено архаїчний звук [ы] як безпосередній продовжувач праслов'янського («мити» → «мыти», «дрова» → «дрыва»); характерні вокальні зсуви у закритих складах ([o] → [y]: «кінь» → «кунь»); запозичені частки угорського і румунського походження. Закарпатський - найскладніший для автоматичної нормалізації: велика лінгвістична відстань від норми і мала кількість ручних паралельних даних утруднюють навчання.

Суржик є принципово іншим явищем - це не регіональний діалект, а соціолінгвістичний феномен змішаного мовлення, поширений у центральних, східних і південних регіонах України. Носії суржику змішують лексичні, морфологічні та фонетичні елементи з обох мов, часто без чіткої регіональної прив'язки [6.]. Автоматичне розпізнавання суржику розглянуто в [13.]. Типові ознаки: лексичні заміни (але → но, дуже → очень, зупинка → остановка), зміна закінчень інфінітива (-ти → -ть: «робити» → «робить»), сполучникові кальки (якщо → если). Оскільки суржик не має єдиної фонологічної системи, його нормалізація потребує покриття широкого спектра лексичних і морфологічних замін.

### ***1.1.3 Наявні ресурси та підходи до синтетичного розширення даних***

Для задачі виправлення граматичних помилок в українській мові сформувалися перші систематичні ресурси. UA-GEC [14.] - перший публічний корпус для граматичної корекції та нормалізації флюентності в українській мові, що охоплює анотацію за типами помилок і дає базу для порівняльного аналізу методів. Багатомовний корпус OmniGEC [8.] розширив задачу на міжмовний рівень, запропонувавши срібну розмітку для кількох мов, включаючи українську. Інструкційно-донавчена модель «Співавтор» [12.] охоплює виправлення помилок, перефразування, спрощення та покращення стилю в українських текстах і є одним із ключових джерел навчальних прикладів у цій роботі.

Діалектна нормалізація перетинається з цими задачами в частині вибору формату даних і методів навчання, але суттєво від них відрізняється: замість «помилки» маємо систематичні трансформації, правильні в межах діалекту. Критичним обмеженням є малоресурсність: для бойківського і закарпатського діалектів ручних паралельних даних майже не існує, а для суржику відсутній усталений стандарт розмітки.

Стандартним рішенням у малоресурсних сценаріях є синтетичне розширення корпусу. Дослідження синтетично згенерованих даних для українського виправлення граматичних помилок [2.] підтверджує, що якість синтетики суттєво залежить від правил навмисного введення відхилень і що моделі, навчені на синтетиці, можуть конкурувати з моделями на ручних даних за достатньої різноманітності прикладів. У цій роботі застосована схема «норма → нестандарт»: за відправну точку береться правильне літературне речення, після чого генерується його діалектний або суржиковий варіант. Перевага такого напрямку

полягає в тому, що еталон гарантовано коректний від початку. Для суржику застосовано два незалежні підходи: правилловий через морфологічний аналізатор і генеративний за допомогою BMM; для регіональних діалектів - лише генеративний із лінгвістичними правилами.

#### ***1.1.4 Сучасні україномовні моделі та найближчі суміжні роботи***

Напрямок адаптації сучасних нейронних архітектур до української мови активно розвивається. Робота «Bytes to Borsch» [7.] дослідила донавчання моделей Gemma і Mistral для покращення україномовної репрезентації і показала, що цілкове донавчання навіть порівняно невеликого обсягу помітно підвищує якість для задач розуміння і генерації. Відкрита україномовна BMM LapaLLM [10.] побудована на базі Gemma-3-12B і оптимізована під українську через адаптацію токенизатора. MamaLM [11.] - менша за параметрами (4 млрд), проте за результатами на стандартних тестах конкурентна модель, яка орієнтована на практичне застосування і доступна через відкриту ліцензію. Для задачі підтримки україномовних текстів загалом важливими є також розвиток ресурсної бази мови [5.] та поєднання математичних моделей і машинного навчання для виявлення і виправлення помилок [4.]. Актуальним є і напрям оцінювання якості моделей від ручних ознак до великих мовних моделей [16.].

Найближчою до нашої постановки є робота «Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation» [9.]: у ній запропоновано методологію адаптації BMM до малоресурсних діалектних сценаріїв, зокрема принципи формування паралельних корпусів і способи донавчання. Разом із тим, ця робота зосереджена на задачі перекладу з діалекту на інші мови, а не на нормалізації до єдиного літературного стандарту усередині мови. Крім того, набір досліджуваних різновидів і суржик як окреме явище в ній не охоплені. Запропонована нами система відрізняється: вона спрямована на нормалізацію чотирьох різновидів - гуцульського, бойківського, закарпатського діалектів і суржику - до літературного стандарту і побудована на власному паралельному корпусі, сформованому комбінацією ручної розмітки і синтетичної генерації.

Аналіз літератури засвідчує три ключових висновки: нормалізація діалектів і суржику методично близька до НМП і виправлення граматичних помилок, але потребує спеціалізованих ресурсів; за браку ручних даних синтетичне розширення через правила і BMM є практичним рішенням; для українських діалектів і суржику публічних паралельних корпусів і спеціалізованих моделей

нормалізації до цього часу не існувало.

## 1.2 Аналіз вхідних даних

Для задачі автоматичної нормалізації якість і репрезентативність навчальних даних є визначальними. На відміну від класичного НМП між різними мовами, тут маємо справу з мовними різновидами однієї мови, де межа між «помилкою», «локальною нормою» та «варіантом написання» часто розмита. Тому до корпусу ставляться додаткові вимоги: узгоджені еталони, прозорий поділ на навчальну, валідаційну та тестову вибірки, контроль джерел (ручні чи синтетичні пари) і баланс між чотирма мовними різновидами.

У роботі застосовано комбінований підхід: як опорну базу взято ручні паралельні дані для гуцульського діалекту; для решти діалектів і суржику корпус сформовано синтетично - за допомогою лінгвістичних правил і великої мовної моделі; як джерело різноманітних літературних цільових речень для суржику використано корпус «Співавтор» [12.].

### 1.2.1 Лінгвістична характеристика мовних різновидів

Для побудови ефективного синтетичного генератора і розуміння природи труднощів задачі нормалізації необхідно систематизувати лінгвістичні особливості кожного мовного різновиду.

*Гуцульський діалект.* На рівні голосних: наголошені «а» і «я» регулярно переходять у «є» («як» → «єк», «шапка» → «шєпка», «яблуко» → «єблуко»); «і» в окремих позиціях переходить у «и» («Іван» → «Иван», «іду» → «иду»); «ю» переходить у «у» («вівцю» → «вівцу»). На рівні приголосних: зворотна частка змінюється з «ся» на «си» і може стояти перед дієсловом («він сміється» → «він си сміє»); сибілянти пом'якшуються («чого» → «чьо»); «ч» та «ц» можуть замінювати одна одну («чи» → «ци»); групи «дн», «тн», «лн» спрощуються до «нн» або «н» («передній» → «перенний»). На морфологічному рівні: дієслова третьої особи однини отримують закінчення -єт («знає» → «знаєт», «має» → «маєт»); у минулому часі можливі аналітичні форми з енклітиками («ходив» → «ходив'єм»); прийменник «від» переходить у «вид»; заперечна частка «не» переходить у «ни» («не знаю» → «ни знаю»).

*Бойківський діалект.* Є одним із найархаїчніших в Україні і зберігає риси давньоукраїнської мови. Головна фонологічна риса - розрізнення переднього [и] і заднього [ы], яке зникло в стандартній мові: «бик» → «бык», «тихо» →

«тыхо», «дівки» → «дівкы». Інша характерна риса - перехід [л] у [ľ] у кінці складу: «галка» → «гавка», «кіл» → «ків». На морфологічному рівні: спрощення груп приголосних («бідна» → «бінна», «весілля» → «весіля»); особові енклітики в умовному способі («ходив би» → «ходивбим»); архаїчна форма третьої особи множини «суть»; унікальні числівникові конструкції («дев'ятнадцять» → «двайцять без єнного»); м'яке вимовляння сибілянтів ([ж], [ш], [ч]). Словник бойківського є найобширнішим серед чотирьох різновидів і містить 14 910 записів.

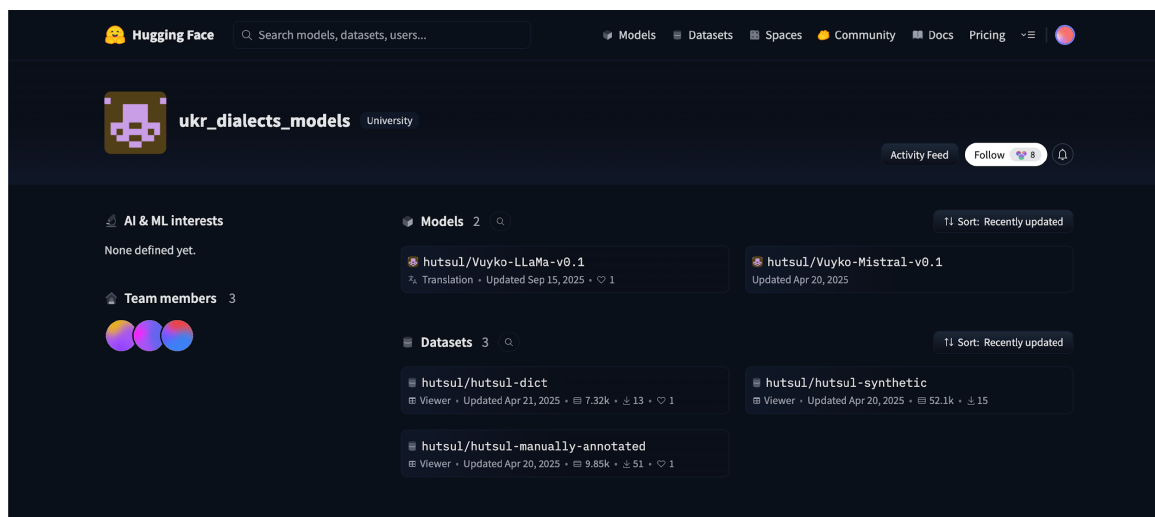
*Закарпатський діалект.* Визначається поєднанням архаїчних праслов'янських рис і запозичень із сусідніх мов. Архаїчний задній голосний [ы] збережено як безпосередній продовжувач праслов'янського звуку: «мити» → «мыти», «синок» → «сынок», «дрова» → «дрыва». Характерні вокальні зсуви у закритих складах: [о] переходить у [у] або [ү] («кінь» → «кунь», «стіл» → «сціл»). З угорської запозичено порівняльну частку «тај» і розділовий сполучник «вад'»; з румунської - «хот'» для поступки. Займенникова система відрізняється від стандартної: демонстративні займенники «тот»/«тота» замість «той»/«та». Велика лінгвістична відстань між закарпатськими формами і літературною нормою обумовлює найнижчу якість нормалізації серед чотирьох різновидів.

*Суржик.* На відміну від діалектів, суржик не є регіональним явищем зі сталою фонологічною системою: кожен носій демонструє власний ступінь і спосіб змішування мов [6.]. Водночас існують повторювані риси: лексичні заміни (але → но, дуже → очень, зупинка → остановка, майже → почті); зміна закінчення інфінітива («робити» → «робить», «ходити» → «ходить»); сполучникові кальки («якщо» → «єслі», «тому що» → «потому шо»); дискурсні маркери («ось» → «вот», «звичайно» → «конєшно»). Стратегія синтетичної генерації суржику полягала у застосуванні 1-3 типових заміन на речення для отримання природного, а не карикатурного змішування.

### **1.2.2 Ресурси для гуцульського діалекту**

Гуцульський діалект обрано відправною точкою, бо для нього доступні найбільш повні ресурси: вручну анотований паралельний корпус, синтетично згенерований корпус і діалектний словник (рис. 1.1). Ручний корпус містить 9 853 пари «гуцульська форма → літературний відповідник» і є основою для формування тестової вибірки - саме на цих даних метрики відображають реальну якість

системи, а не якості генератора синтетики.



**Рис. 1.1:** Гуцульські ресурси на платформі Hugging Face: датасети та допоміжні матеріали, що використовувались як вихідні джерела.

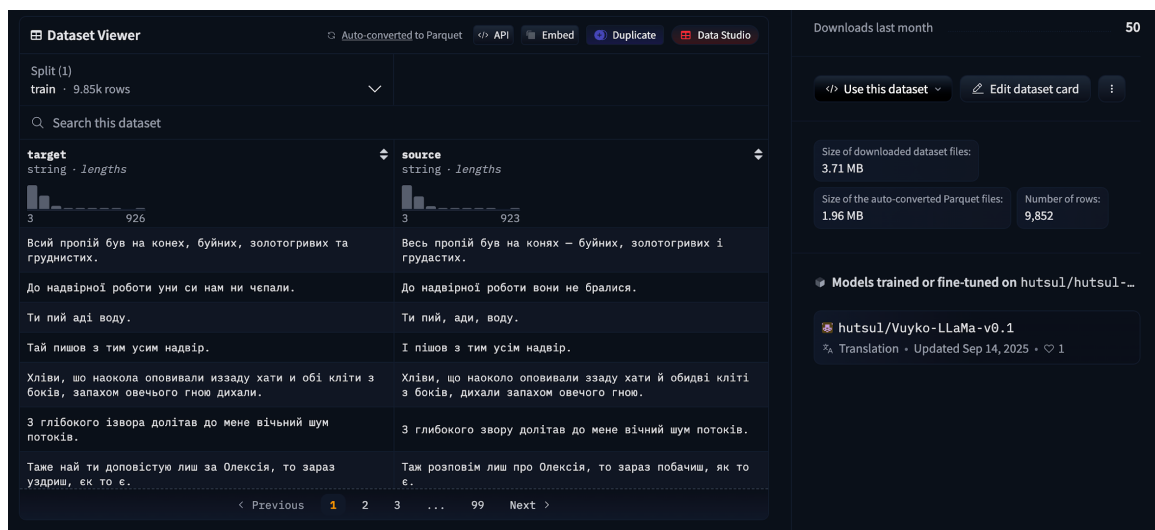
Ручний корпус доповнено синтетично згенерованими парами: за допомогою моделі Mistral Small 24B з лінгвістичними правилами гуцульського діалекту отримано 19 841 пару. Таким чином, загальний обсяг гуцульської підвибірki в навчальному наборі склав 29 438 прикладів.

Діалектний словник виконував кілька функцій: слугував джерелом лексичних відповідників під час синтетичної генерації, обмежував простір допустимих діалектних заміни і використовувався для валідації згенерованих пар - якщо у вхідному тексті були діалектні слова, перевірялась їх наявність у словнику. Словник містить 7 320 записів у форматі «діалектна форма - стандартна форма - словникова лема».

На рис. 1.2 показано приклад вмісту вручну анотованого корпусу: кожен рядок містить вхідний діалектний текст і відповідний літературний еталон. У задачі нормалізації один діалектний вираз може мати кілька допустимих літературних варіантів, тому ручна розмітка фіксує один узгоджений еталон.

### **1.2.3 Корпус «Співавтор» як джерело літературної норми**

Для суржику еталонні речення літературної норми взято з корпусу «Співавтор» (рис. 1.3) - набору пар редагування і перефразування українською мовою, що охоплює виправлення граматичних помилок, перефразування і спрощення тексту [12.]. У контексті цієї роботи «Співавтор» використовувався не як діалектний ресурс, а як пул стилістично природних літературних речень: з нього відбирались цільові речення, на основі яких генерувався суржиковий варіант.



**Рис. 1.2:** Вміст вручну анотованого паралельного корпусу для гуцульського діалекту: стовпці вхідного тексту та еталонного відповідника.

Практична логіка така: обирається літературне речення  $y$  зі «Співавтора», після чого генерується суржиковий варіант  $x$ . Таким чином пара  $(x, y)$  гарантовано має якісний еталон. Перевага такого підходу полягає ще й у тому, що «Співавтор» містить широкий спектр природних речень із різних доменів - новини, побутові тексти, публіцистика, - що забезпечує доменну різноманітність навчальних прикладів для суржику.

#### 1.2.4 Словникова база мовних різновидів

Для кожного з чотирьох мовних різновидів зібрано словники відповідників між діалектними і стандартними формами. Словники виконували подвійну роль у системі: слугували джерелом прикладів для шаблонів запитів до ВММ та засобом контролю якості згенерованих пар. Основні характеристики словників наведено в таблиці 1.

**Таблиця 1:** Словники діалектних відповідників

| Мовний різновид | Записів | Формат запису                           |
|-----------------|---------|-----------------------------------------|
| Гуцульський     | 7 320   | Діалектна форма, стандартна форма, лема |
| Бойківський     | 14 910  | Діалектна форма, стандартна форма, лема |
| Закарпатський   | 7 469   | Діалектна форма, стандартна форма, лема |
| Суржик          | 570     | Суржикова форма, стандартна форма       |

Бойківський словник є найбільшим (14 910 записів), що відображає зна-



**Datasets:** [grammarly/spivavtor](#) like 5 Follow Grammarly 200 Dataset card

Split (2)  
train · 62.8k rows

Search this dataset

| id      | src                                                                                                                       | tgt                                                                                       | task             |
|---------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------|
| int64   | string · lengths                                                                                                          | string · lengths                                                                          | string · classes |
| 106.28k | 99*176 34.3%                                                                                                              | 77*153 29.8%                                                                              | gec 44.5%        |
| 1       | Перефразуйте: Кайс став одержимий нещодавно усвідомленою вразливістю.                                                     | Кайс був одержимий своєю нововиявленою вразливістю.                                       | paraphrase       |
| 2       | Перепишіть речення іншими словами: І вона більше не думатиме про Джонні Сміта та його...                                  | Джонні Сміт і його дивна чарівна посмішка, про яку він ніколи більше не згадав.           | paraphrase       |
| 3       | Виправити граматику: З простими, там, поїсти, поспати не важко.                                                           | З простими, там, поїсти, поспати не важко.                                                | gec              |
| 4       | Виправте всі граматичні помилки: Тоді як ДНК має дві спіралі – інформаційний ряд...                                       | Тоді як ДНК має дві спіралі – інформаційний ряд з'єднаний з комплементарним йому іншим... | gec              |
| 5       | Зробіть цей текст легше для розуміння: Ви в дуже серйозній біді, містере Штайнфелд.                                       | Містере Штайнфелд, у вас великі проблеми.                                                 | simplification   |
| 6       | Виправте зв'язність в цьому тексті: англіканство вперше прийшло до Бразилії на початку 19 століття, коли португальська... | Англіканство вперше прибуло до Бразилії на початку 19 століття, коли португальська...     | coherence        |
| 7       | Покращте зв'язність тексту: пісні у стилі блек-метал часто відрізняються від...                                           | Блек-метал-пісні часто відрізняються від традиційної структури пісень і часто не...       | coherence        |
| 8       | Виправити граматику: Іван побачив сірий берет в гуші на початку Великої Микитської...                                     | Іван побачив сірий берет у гуші на початку Великої Микитської, але Герцена.               | gec              |

< Previous 1 2 3 ... 628 Next >

**Рис. 1.3: Приклад вмісту корпусу «Співавтор»: вхідний і відредагований текст із міткою типу завдання.**

чну лексичну відстань між бойківськими формами і стандартом. Словник суржику суттєво менший (570 записів), оскільки основні відхилення суржику є морфологічними і фонетичними (зміна закінчень, фонетичні кальки), а не лексичними замінами: для таких перетворень застосовується морфологічний аналізатор, а не словникова таблиця. Під час синтетичної генерації до кожного запиту включалось 3-5 прикладів із відповідного словника, що допомагало моделі дотримуватися лінгвістичного стилю конкретного діалекту.

### 1.2.5 Синтетична генерація даних

Для більшості мовних різновидів ручних паралельних даних критично мало, тому синтетичне розширення є необхідним. У роботі застосовано підхід «норма → нестандарт»: за вхідний еталон  $y$  береться літературне речення, а нестандартний варіант  $x$  генерується. Це зручно, бо якість еталону гарантована від початку.

*Правилова генерація суржику.* Реалізовано двофазову заміну. На першій фазі здійснюється фразова заміна: за допомогою регулярних виразів виявляються найдовші збіги із словником багатослівних суржикових конструкцій і за-

мінюються їхніми еквівалентами (наприклад, «тому що» → «потому що»). Ця фаза обробляє контекстні сполучення до розбиття тексту на окремі слова. На другій фазі виконується морфологічна заміна на рівні слів: кожне слово лематизується через бібліотеку морфологічного аналізу `rumorphy2`, і якщо словникова лема входить до списку замін суржику, відповідне слово замінюється з урахуванням граматичних категорій - відмінку, числа і роду. Такий підхід забезпечує не просто лексичну, а морфологічно узгоджену заміну і дав 20 911 прикладів.

*Генерація за допомогою ВММ.* Застосована для всіх чотирьох мовних різновидів. Для кожного діалекту підготовлено окремий системний запит, що містить лінгвістичний опис типових рис, 3-5 демонстраційних пар зі словника та інструкцію щодо кількості і стилю замін. Велика мовна модель Mistral Small 24B (4-бітна квантизація для локальної інференції) обробляла пакети по 5-10 речень і повертала діалектні варіанти. Приклад шаблону запиту зображено на рис. 1.4.

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."  
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."  
* Assistant: "...нормалізований текст..."
```

**Рис. 1.4: Приклад шаблону запиту для синтетичної генерації діалектних пар: інструкція, правила діалекту і демонстраційні приклади.**

Контроль якості синтетичних пар проводився після кожного пакету: видалялись дублікати (однакові вхідні або еталонні тексти), відкидались надто короткі (менше трьох слів) та надто довгі пари, перевірялась наявність діалектних слів зі словника у вхідному тексті, вибірково проводився ручний перегляд для оцінки типових артефактів генерації - змішування мов, незавершених речень, залишків форматування у відповіді.

### **1.2.6 Формування та структура фінального корпусу**

Після генерації всіх складових дані об'єднано в єдиний корпус і розділено на три вибірки. Загальна структура наведена в таблиці 2.

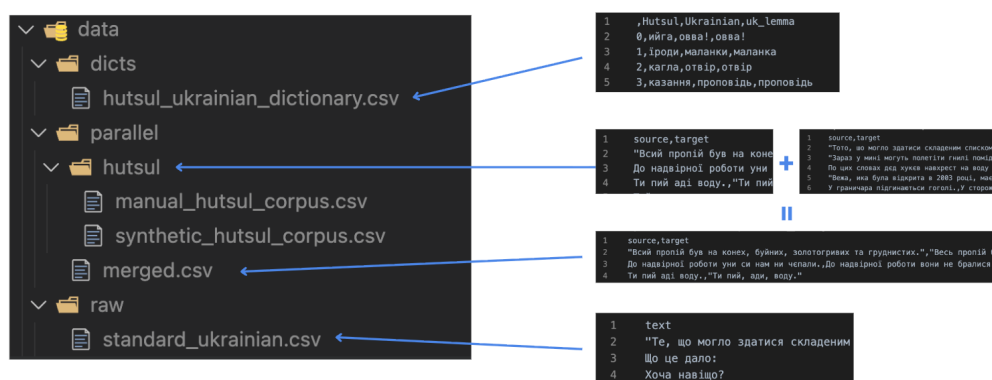
Після формування загального об'єднаного набору 90% прикладів виділено у навчальну вибірку (125 615 прикладів), 10% - у валідаційну (13 756 прикладів). Тестова вибірка з 200 прикладів (по 50 на кожен мовний різновид) сформована виключно з вручну анотованих пар, що забезпечує об'єктивність оцінювання і виключає вплив якості синтетичного генератора на тестові метрики.

Ключовий принцип організації: тестова і валідаційна вибірки формую-

**Таблиця 2: Склад паралельного корпусу по мовних різновидах**

| Різнovid      | Ручні        | LLM            | Правилкові    | Разом          |
|---------------|--------------|----------------|---------------|----------------|
| Гуцульський   | 9 853        | 19 841         | -             | 29 694         |
| Бойківський   | -            | 29 754         | -             | 29 754         |
| Закарпатський | -            | 29 605         | -             | 29 605         |
| Суржик        | -            | 29 612         | 20 911        | 50 523         |
| <b>Разом</b>  | <b>9 853</b> | <b>108 812</b> | <b>20 911</b> | <b>139 576</b> |

тється лише з ручних пар, синтетика використовується виключно в навчальній вибірці. Це дозволяє уникнути переоцінки якості на штучних прикладах і отримувати метрики, близькі до реального застосування. Структуру директорій репозиторію і розміщення файлів корпусу показано на рис. 1.5.



**Рис. 1.5: Структура директорій проєкту: словники, ручні корпуси, синтетичні дані та фінальні вибірки для навчання, валідації і тестування.**

### 1.3 Висновки до розділу 1

У розділі проаналізовано предметну галузь, розглянуто суміжні дослідження та сформовано корпус для навчання системи.

Нормалізація діалектів і суржику є задачею на межі між НМП і виправленням граматичних помилок: тут присутні як систематичні лексичні заміни, так і фонетичні та морфологічні трансформації регулярного характеру. Аналіз літератури показав: для малоресурсних сценаріїв синтетичне розширення даних і донавчання ВММ є ефективними підходами, а для українських діалектів і суржику до цього часу не існувало спеціалізованих паралельних корпусів і моделей нормалізації.

Чотири досліджуваних різновиди суттєво відрізняються за природою відхилень: гуцульський і бойківський є регіональними діалектами з архаїчними

фонологічними рисами, закарпатський демонструє контактний вплив суміжних мов, суржик є соціолінгвістичним явищем змішаного мовлення.

Зібрано паралельний корпус із 139 576 прикладів загалом, з яких 125 615 увійшли до навчальної вибірки. Для гуцульського діалекту використано ручний корпус (9 853 пари) і словник (7 320 записів). Для бойківського і закарпатського словники містять 14 910 і 7 469 записів відповідно. Суржиковий корпус отримано поєднанням правилowego підходу (20 911 прикладів) і генерації за допомогою ВММ (29 612). Тестові вибірки сформовано виключно з ручних пар, що забезпечує надійне оцінювання якості у наступних розділах.

## 2 ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

Розділ охоплює математичну постановку задачі нормалізації, опис двох реалізованих архітектур, обґрунтування вибору моделей і конфігурацій навчання, а також метрики оцінювання. Загальну схему системи показано на рис. 2.1.

### 4-Phase ML Pipeline:

Phase 1: Data Generation  
Input: Standard Ukrainian text  
Tool: OpenRouter/Local  
Output: Parallel corpus (CSV: source → target)

Phase 2: Model Training  
Input: Parallel corpus (data/parallel/merged.csv)  
Tool: PyTorch • Transformers  
Model: Encoder-Decoder (umT5) or Decoder-Only (Llama/Gemma/Lapa/Mamay)  
Output: Fine-tuned checkpoint (models/)

Phase 3: Evaluation  
Input: Test set (data/parallel/eval.csv)  
Tool: PyTorch • Transformers  
Metric: BLEU/chrF++/TER  
Output: predictions.txt + references.txt

Phase 4: Inference  
Input: Dialect/surzhyk text  
Tool: PyTorch • Transformers • Gradio  
Output: Normalized Ukrainian text

**Рис. 2.1: Узагальнена схема системи: (а) базова модель «гуцульський → літературна», (б) мультидіалектний і суржиковий етап на основі MambaLM, (в) демонстраційний інтерфейс.**

### 2.1 Аналіз підходів до нормалізації

#### 2.1.1 Нормалізація як задача перетворення тексту

Нормалізацію діалектних форм і суржику розглядаємо як кероване перетворення тексту: необхідно зберегти зміст, але змінити форму відповідно до літературної норми. Такі перетворення охоплюють лексичні заміни (діалектизми), варіативну орфографію, фонетично мотивовані написання, а у суржику - ще й перемикання між мовами на рівні лексики.

Формально маємо вхідний рядок  $x$  (діалект або суржик) та бажаний вихідний рядок  $y$  (літературна українська). Мета - побудувати модель  $f_\theta$ , що апроксимує відображення:

$$f_\theta : x \mapsto y$$

З погляду статистичного моделювання це еквівалентно оцінюванню умовного розподілу  $p_\theta(y|x)$ .

Практичні особливості задачі:

- *Часткова тотожність*: значна частина лексем у  $x$  може збігатися з  $y$ , особливо у суржику;
- *Асиметрія*: «виправляти занадто багато» ризикує спотворити зміст, «виправляти занадто мало» - не виконати нормалізацію;

- *Багатоваріантність норми*: одна діалектна форма може відповідати кільком літературним парафразам;
- *Низькоресурсність*: для більшості діалектів немає великих вручну розмічених паралельних корпусів.

### **2.1.2 Правиліві підходи та їх обмеження**

Правилова нормалізація використовує словники відповідників, регулярні правила перетворень та евристики. Її перевага - контрольованість: якщо відомі правила заміни, можна гарантувати певну поведінку. Для словників, описаних у підрозділі 1.2.4, кожен запис задає відповідність між нестандартною і літературною формами. Попри це правилівий підхід має системні обмеження:

- складно охопити всю лексичну варіативність без великого словника;
- важко коректно обробляти контекстні значення (омонімія, полісемія);
- правила погано підходять для довгих залежностей та синтаксичних перетворень;
- якість деградує при зміні домену (побутові повідомлення, конспекти, оголошення).

У цій роботі правиліві ресурси відіграють допоміжну роль: (а) для побудови синтетичних навчальних даних; (б) для підказок під час LLM-генерації синтетики; (в) як інструмент аналізу помилок.

### **2.1.3 Нейромережеві підходи: кодер-декодерна модель і декодерна ВММ**

У роботі послідовно застосовано дві парадигми нейромережевого підходу.

**Кодер-декодерна модель для базової перевірки на гуцульському. Переваги:**

- стабільне навчання на паралельних даних;
- природна постановка: вхідний текст → нормалізований текст;
- компактний розмір ( $\approx 300$  млн параметрів) дозволяє повне донавчання на одному ГП.

**Декодерна ВММ з інструкційним форматом для всіх діалектів і суржику. Переваги:**

- одна модель обробляє всі чотири різновиди через маркер у запиті;
- використання мовних знань моделі ( $\approx 4$  млрд параметрів) для кращого ро-

зуміння контексту;

- природне розширення на нові задачі без зміни архітектури.

Для декодерної парадигми обрано MatayLM - відкриту україномовну ВММ на базі Gemma-3-4B, оптимізовану для інструкційного виконання задач [11.].

## 2.2 Постановка задачі та математична модель

### 2.2.1 Формальна постановка нормалізації

Нехай  $\Sigma$  - алфавіт символів (або множина лексем після токенизації). Вхідний текст:

$$x = (x_1, x_2, \dots, x_{|x|}), \quad x_i \in \Sigma$$

вихідний текст (літературна норма):

$$y = (y_1, y_2, \dots, y_{|y|}), \quad y_j \in \Sigma$$

Задача - знайти параметри  $\theta$ , що максимізують правдоподібність навчальних пар  $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$ :

$$\theta^* = \arg \max_{\theta} \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)}) \quad (1)$$

У практичній реалізації мінімізується негативний лог-правдоподібний - крос-ентропія:

$$L(\theta) = - \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)}) \quad (2)$$

### 2.2.2 Формат даних для двох архітектур

**Кодер-декодерна модель.** Дані зберігаються у вигляді пар «вхідний текст - еталон». Після токенизації отримуємо послідовності індексів  $x = (x_1, \dots, x_T)$ ,  $y = (y_1, \dots, y_{T'})$ . Модель навчається прогнозувати  $y_t$  з урахуванням усього вхідного тексту та попередніх виходів:

$$p_{\theta}(y|x) = \prod_{t=1}^{T'} p_{\theta}(y_t | x, y_{<t}) \quad (3)$$

**Декодерна ВММ.** Задачу представляємо як інструкцію з трьома ролями: «система», «користувач» і «асистент» (рис. 2.2). Вхідний текст передається у ролі «користувач», нормалізована форма - у ролі «асистент». Об'єднана послідовність лексем  $z = (z_1, \dots, z_{|z|})$  включає всі три ролі. Функція втрат обчислюється лише на лексемах відповіді «асистент»:

$$L_{mask} = - \sum_{t=1}^{|z|} m_t \log p_{\theta}(z_t | z_{<t}) \quad (4)$$

де  $m_t \in \{0, 1\}$  - маска (1 лише на позиціях відповіді «асистент»).

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."
* Assistant: "...нормалізований текст..."
```

**Рис. 2.2:** Приклад шаблону інструкції для декодерної ВММ: ролі системи, користувача та асистента, де вставляється маркер діалекту.

### 2.2.3 Цільова функція: крос-ентропія і згладжування міток

Поелементна крос-ентропія для кодер-декодерної моделі:

$$L_{CE} = - \frac{1}{T'} \sum_{t=1}^{T'} \log p_{\theta}(y_t | x, y_{<t}) \quad (5)$$

Для umt5-base застосовано згладжування міток із коефіцієнтом  $\alpha = 0.1$ . Ідея: замість жорсткого розподілу (1 для правильної лексеми, 0 для решти) навчаємо на розм'якшеному розподілі:

$$L_{smooth} = (1 - \alpha) \cdot L_{CE} + \frac{\alpha}{|V|} \sum_{v \in V} \log p_{\theta}(v | x, y_{<t}) \quad (6)$$

де  $V$  - словник моделі,  $|V|$  - його розмір. Згладжування міток запобігає надмірній впевненості моделі та покращує узагальнення на нові приклади.



## 2.3 Математичні основи трансформера

### 2.3.1 Механізм уваги

Основою обох архітектур є трансформер із механізмом уваги. Нехай на вході є матриця представлень  $H \in \mathbb{R}^{T \times d}$ . Обчислюються запит, ключ і значення:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V$$

де  $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$  - навчувані матриці проєкцій.

Увага:

$$\text{Увага}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (7)$$

Масштабування на  $\sqrt{d_k}$  стабілізує градієнти при великих розмірностях. Багатоголова увага поєднує  $h$  незалежних голів:

$$\text{БГУ}(H) = \text{Concat}(\text{голова}_1, \dots, \text{голова}_h)W_O \quad (8)$$

де  $\text{голова}_i = \text{Увага}(HW_{Q_i}, HW_{K_i}, HW_{V_i})$ .

### 2.3.2 Позиційне кодування, нормалізація шарів і прямозв'язний блок

Трансформер не має вбудованого поняття порядку - для збереження позиційної інформації до вхідних представлень додається позиційне кодування. У класичній синусоїдальній схемі:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (9)$$

де  $pos$  - позиція лексеми в послідовності,  $i$  - індекс розмірності,  $d$  - розмірність моделі. Такий підхід дозволяє розрізняти відносні відстані між лексемами без введення додаткових параметрів. У сучасних архітектурах, зокрема Gemma-3 (основа MambaLM), замість синусоїдального застосовується ротаційне позиційне кодування (RoPE), яке краще узагальнюється на довші послідовності.

Кожен підшар трансформера (увага або прямозв'язний блок) доповнюється залишковим з'єднанням і нормалізацією шарів:

$$x' = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (10)$$

де  $\text{Sublayer}$  - або увага, або прямозв'язний блок. Залишкове з'єднання  $x +$

$\text{Sublayer}(x)$  полегшує прямий потік градієнта через глибоку мережу і запобігає деградації навчання при великій кількості шарів.

Прямозв'язний блок (FFN), розташований після шару уваги в кожному блоці трансформера:

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (11)$$

де  $W_1 \in \mathbb{R}^{d \times d_{ff}}$ ,  $W_2 \in \mathbb{R}^{d_{ff} \times d}$ , а  $d_{ff}$  - внутрішня розмірність (зазвичай  $4d$ ). Нелінійність ReLU ( $\max(0, \cdot)$ ) дозволяє FFN зберігати фактичні знання про мову, тоді як механізм уваги відповідає за зв'язки між позиціями послідовності.

### 2.3.3 Кодер-декодерна архітектура

Кодер перетворює вхідну послідовність  $x$  у контекстні представлення  $E \in \mathbb{R}^{T \times d}$  за допомогою стеку шарів: самоувага  $\rightarrow$  нормалізація  $\rightarrow$  прямозв'язний блок  $\rightarrow$  нормалізація. Декодер генерує вихідну послідовність  $y$ , використовуючи:

- самоувагу по вже згенерованих лексемах  $y_{<t}$ ;
- перехресну увагу до контекстних представлень  $E$ .

Умовна авторегресійна генерація:

$$p_{\theta}(y|x) = \prod_{t=1}^{T'} p_{\theta}(y_t|x, y_{<t}) \quad (12)$$

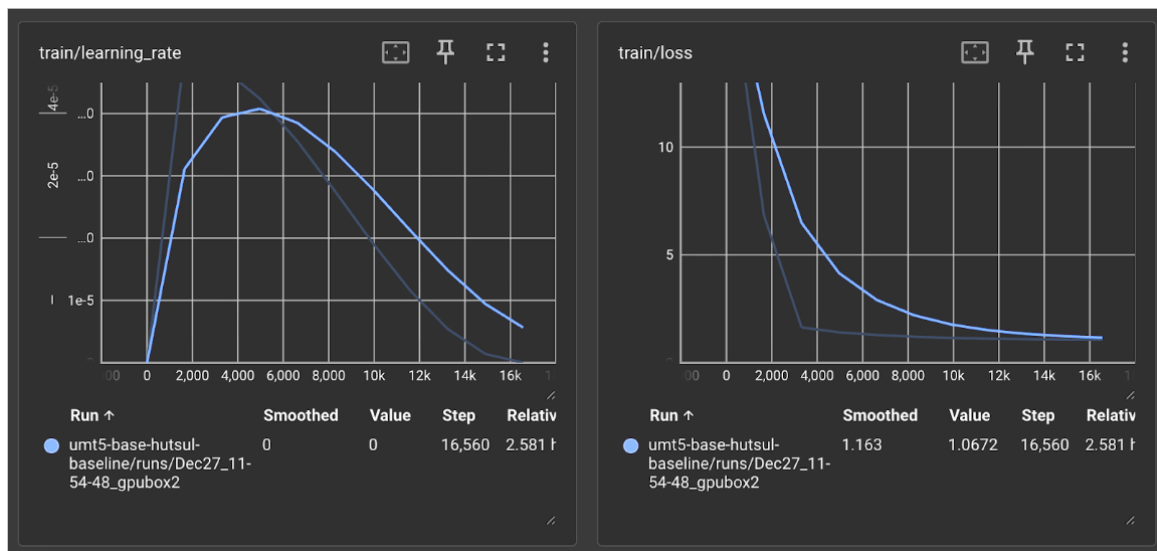
Базову кодер-декодерну модель umt5-base навчено спочатку на гуцульському діалекті. Графік навчання наведено на рис. 2.3, результати оцінювання - на рис. 2.4.

### 2.3.4 Авторегресійна декодерна модель

Декодерна ВММ моделює об'єднану послідовність лексем  $z$  (система + запит + відповідь) авторегресивно:

$$p_{\theta}(z) = \prod_{t=1}^{|z|} p_{\theta}(z_t|z_{<t}) \quad (13)$$

Усі лексеми обробляються в одному прямому проходженні, але функція втрат (4) оптимізується лише за лексемами відповіді «асистент». Завдяки цьому модель залишається загальною мовною моделлю, але спеціалізується на кон-



**Рис. 2.3: Графік навчання базової моделі *umt5-base* на гуцульському діалекті: темп навчання та функція втрат.**

кретній задачі нормалізації.

## 2.4 Вибір та обґрунтування моделей

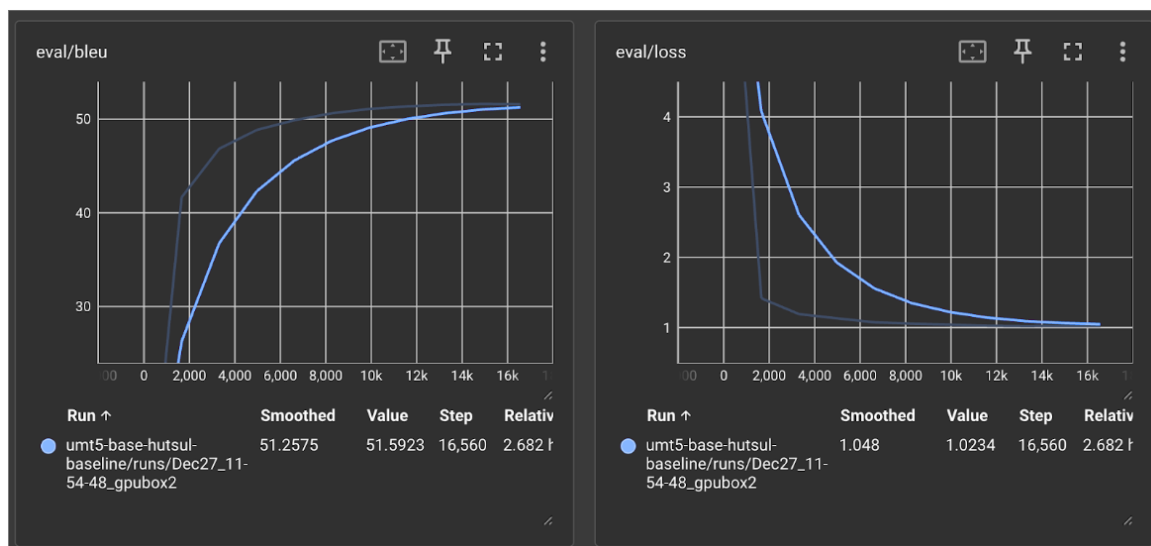
### 2.4.1 Кодер-декодерна модель *umt5-base*

Для базової перевірки та кодер-декодерного підходу обрано google/umt5-base - багатомовну T5-модель ( $\approx 300$  млн параметрів), натреновану на 100+ мовах включно з українською [3.]. Причини вибору:

1. Попереднє тренування охоплює значний обсяг українського тексту.
2. Кодер-декодерна архітектура природно підходить для нормалізації як «перекладу».
3.  $\approx 300$  млн параметрів дозволяє повне донавчання на одному ГП.
4. Відтворюваність: модель відкрита, розміщена у сховищі Hugging Face.

Перший етап - навчання лише на гуцульському діалекті - слугував базовою лінією. Гуцульський обрано через найбільший обсяг ручної розмітки (9 853 пари) та наявність детального словника. Мета: перевірити паралельні дані, конвеєр токенизації і навчання, а також встановити відправну точку для порівняння при переході до мультидіалектної задачі. Графік навчання (рис. 2.3) показує стабільне зниження функції втрат, BLEU на валідаційній вибірці стабілізується приблизно на рівні 51, що підтверджує коректність реалізованого конвеєра.

Другий етап - мультидіалектне навчання - об'єднує всі чотири різновиди в одному корпусі ( 125 тис. прикладів). Спільна модель вчиться вирізняти



**Рис. 2.4:** Результати оцінювання базової моделі *umt5-base*: метрика BLEU (ліворуч) та функція втрат на валідаційній вибірці (праворуч) під час навчання.

між діалектами за допомогою системного маркера в запиті, переданому разом із текстом. Кодер-декодерна архітектура виграє від того, що кодер обробляє вхідний текст паралельно і незалежно від довжини виходу - це стабілізує навчання на різномірних парах (суржик з мінімальними замінами vs. гуцульський з фонетичними змінами).

#### 2.4.2 Декодерна ВММ MatayLM

Для мультидіалектного підходу і суржику обрано MatayLM - декодерну ВММ на базі Gemma-3-4B ( $\approx 4$  млрд параметрів), адаптовану для української [11.]. Причини вибору над іншими варіантами:

1. **Розмір:** 4 млрд параметрів (проти, наприклад, 12 млрд у Gemma-3-12B) дозволяє QLoRA-донавчання на одному ГП із 24 ГБ відеопам'яті.
2. **Українська мова:** модель оптимізована для задач на українській, зокрема для інструкційного виконання.
3. **Відкрита ліцензія:** результати можна публікувати і відтворювати.
4. **Інструкційна версія:** підтримує чат-шаблон (система/користувач/асистент), що спрощує навчання одночасно на чотирьох різновидах.

Інструкційний формат дозволяє явно вказувати тип завдання через маркер у ролі «система»: нормалізація гуцульського, бойківського, закарпатського або

суржику. Таким чином одна модель розв’язує чотири окремі підзадачі.

### 2.4.3 Параметро-ефективне донавчання: LoRA та QLoRA

Повне донавчання моделі з мільярдами параметрів вимагає значних обчислювальних ресурсів. Для MatryLM застосовано LoRA - метод низькорангової адаптації (рис. 2.5).

Нехай у трансформері є матриця ваг  $W \in \mathbb{R}^{d \times k}$ . LoRA додає низькорангову поправку:

$$W' = W + \Delta W = W + BA \quad (14)$$

де  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times k}$ ,  $r \ll \min(d, k)$ . Кількість тренованих параметрів скорочується з  $dk$  до  $r(d + k)$ . При  $r = 16$  для шару розміром  $4096 \times 4096$ : замість  $\approx 16.8$  млн тренується лише  $\approx 131$  тис. параметрів.

Масштабуючий коефіцієнт регулює величину поправки:

$$s = \frac{\alpha}{r} = \frac{32}{16} = 2 \quad (15)$$

Цільові модулі LoRA: проєкції уваги (q\_proj, k\_proj, v\_proj, o\_proj) та прямозв’язні шари (gate\_proj, up\_proj, down\_proj).

QLoRA доповнює LoRA 4-бітним квантуванням базової моделі у форматі NF4 (Normal Float 4) із збереженням адаптерів у bfloat16. Подвійне квантування (Double Quantization) квантує і самі коефіцієнти квантування, зменшуючи споживання ОП ще приблизно на 0.37 біт на параметр. Разом QLoRA дозволяє донавчати 4B-модель у межах 24 ГБ відеопам’яті.

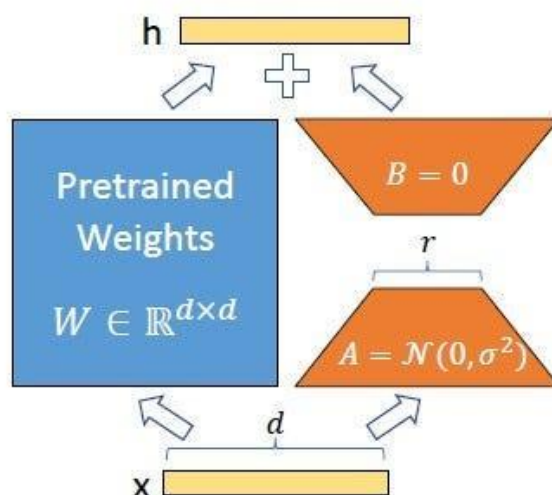
## 2.5 Гіперпараметри та конфігурація навчання

### 2.5.1 Донавчання *umt5-base*

Усі гіперпараметри донавчання *umt5-base* наведено у табл. 3.

Вибір темпу навчання  $5 \times 10^{-5}$  відповідає рекомендованому діапазону для повного донавчання T5-сімейства: вищі значення ризикують руйнуванням попередньо навчених представлень (catastrophic forgetting), нижчі уповільнюють збіжність на малих корпусах. Косинусний спадаючий планувальник забезпечує плавне зниження темпу навчання до нуля до кінця навчання, уникаючи різких стрибків.

Кількість епох (40) дещо більша, ніж у стандартних задачах нейронного машинного перекладу, де зазвичай достатньо 10-15 епох. Нормалізація діа-



**Рис. 2.5: Ілюстрація LoRA: вихідні ваги  $W$  заморожені, а низькорангова поправка  $BA$  додається у шарах уваги та прямозв'язних блоках.**

лектів має меншу кількість навчальних прикладів і потребує більше оновлень для якісного засвоєння. Ранній зупин із терпінням 5 епох за метрикою BLEU на валідаційній вибірці запобігає перенавчанню: якщо BLEU не покращується протягом 5 послідовних епох, навчання припиняється і зберігається найкраща контрольна точка.

8-бітний AdamW через BitsAndBytes зменшує споживання відеопам'яті оптимізатором приблизно у 4 рази порівняно зі стандартним 32-бітним AdamW, не впливаючи помітно на якість. Заповнення до кратного 8 лексем прискорює матричні операції на ГП завдяки вирівнюванню пам'яті.

### **2.5.2 Доновчання MatayLM (QLoRA)**

Повна конфігурація донавчання MatayLM наведена у табл. 4.

Параметри генерації під час оцінювання MatayLM: температура = 0.1, top-k = 25, top-p = 1.0, максимальна кількість нових лексем = 256, штраф за повтори = 1.1.

Темп навчання  $2 \times 10^{-4}$  для QLoRA вищий, ніж для повного донавчання umt5-base. Це обгрунтовано тим, що LoRA-адаптери ініціалізуються з нуля (матриця  $B$  - нулями,  $A$  - малим гаусівським шумом) і не мають попередньо навченого стану, який можна пошкодити. Водночас заморожені ваги базової моделі не змінюються, тому ризик руйнування мовних знань відсутній.

Три епохи достатні для задачі нормалізації завдяки тому, що базова модель MatayLM вже добре знає українську - залишається лише сформувати

**Таблиця 3: Гіперпараметри донавчання `umt5-base`**

| Параметр                        | Значення                           |
|---------------------------------|------------------------------------|
| Базова модель                   | <code>google/umt5-base</code>      |
| Кількість параметрів            | $\approx 300$ млн                  |
| Кількість епох                  | 40                                 |
| Розмір пакету (накопичення: 4)  | 4 (ефективний: 16)                 |
| Темп навчання                   | $5 \times 10^{-5}$                 |
| Планувальник темпу              | косинусний спад                    |
| Кроки прогріву                  | $\min(1000, 0.1 \times N_{steps})$ |
| Оптимізатор                     | AdamW 8-bit (BitsAndBytes)         |
| Згладжування міток ( $\alpha$ ) | 0.1                                |
| Спадання ваг                    | 0.1                                |
| Числовий формат                 | <code>bfloat16</code>              |
| Макс. довжина послідовності     | 256 лексем                         |
| Ранній зупин                    | терпіння 5 епох (за BLEU)          |
| Заповнення                      | до кратного 8 лексем               |

стиль відповіді. Більша кількість епох ризикує переспеціалізувати модель під конкретні приклади навчальної вибірки, погіршуючи узагальнення на нових текстах.

Пакування послідовностей (`packing=True`) об'єднує кілька коротких прикладів в одну послідовність максимальної довжини 512 лексем. Це знижує частку заповнювальних лексем (`padding`) у пакеті та підвищує ефективне використання ГП: замість навчання на переважно порожніх пакетах модель отримує більше фактичного навчального сигналу за той самий час.

## 2.6 Метрики оцінювання

### 2.6.1 BLEU

BLEU (Bilingual Evaluation Understudy) оцінює схожість згенерованого тексту з еталоном через модифіковану n-грамну точність [16.]:

$$\text{BLEU} = BP \cdot \exp \left( \sum_{n=1}^N w_n \log p_n \right) \quad (16)$$

**Таблиця 4: Гіперпараметри донавчання MamayLM (QLoRA)**

| Параметр                       | Значення                   |
|--------------------------------|----------------------------|
| Базова модель                  | MamayLM-Gemma-3-4B-IT-v1.0 |
| Кількість параметрів           | $\approx 4$ млрд           |
| Кількість епох                 | 3                          |
| Розмір пакету (накопичення: 2) | 1 (ефективний: 2)          |
| Темп навчання                  | $2 \times 10^{-4}$         |
| Планувальник темпу             | косинусний спад            |
| Частка прогріву                | 0.03                       |
| Оптимізатор                    | AdamW 8-bit (Unsloth)      |
| Ранг LoRA ( $r$ )              | 16                         |
| Масштаб LoRA ( $\alpha$ )      | 32                         |
| Відсів LoRA (dropout)          | 0.0                        |
| Квантування                    | NF4, 4-bit, подвійне       |
| Числовий формат                | bfloat16                   |
| Макс. довжина послідовності    | 512 лексем                 |
| Пакування послідовностей       | так                        |

де  $p_n$  - модифікована точність n-грам,  $w_n = 1/N$  - рівні ваги,  $BP$  - штраф за стислість:

$$BP = \begin{cases} 1, & |\hat{y}| > |y| \\ \exp\left(1 - \frac{|y|}{|\hat{y}|}\right), & |\hat{y}| \leq |y| \end{cases} \quad (17)$$

де  $|\hat{y}|$  - довжина згенерованого тексту,  $|y|$  - довжина еталону.

У роботі обчислюється corpus\_bleu із бібліотеки SacreBLEU. Значення BLEU є критерієм збереження найкращої контрольної точки під час навчання umt5-base.

### 2.6.2 chrF++

chrF++ розширює метрику chrF, додаючи до символьних n-грам словесні (параметр word\_order = 2). Базова F-міра:

$$\text{chrF} = \frac{(1 + \beta^2) \cdot \text{chrP} \cdot \text{chrR}}{\beta^2 \cdot \text{chrP} + \text{chrR}} \quad (18)$$

де chrP - символьна точність, chrR - символьна повнота,  $\beta = 2$  (більший акцент на повноту, ніж на точність).

Переваги chrF++ для задачі нормалізації:



- нечутлива до токенизації - оцінює безпосередньо символи;
- краще відображає морфологічну схожість для мов із розгалуженою флексійною системою;
- стабільніша за BLEU на коротких виходах, де n-грамні підрахунки нестійкі.

У роботі chrF++ є основною порівняльною метрикою. Обчислюється як `corpus_chrf(word_order = 2)` у SacreBLEU.

### 2.6.3 TER

TER (Translation Edit Rate) вимірює кількість редагувань, необхідних для перетворення згенерованого тексту на еталон:

$$TER = \frac{\text{кількість редагувань}}{|\text{еталон}|} \quad (19)$$

Типи редагувань: вставка, видалення, заміна та зсув фрагмента. На відміну від BLEU і chrF++, менше значення TER означає кращу якість. Обчислюється як `corpus_ter` у SacreBLEU.

### 2.6.4 Взаємодоповнюваність метрик

Три метрики охоплюють різні аспекти якості і доповнюють одна одну:

- BLEU акцентує на точності n-грамних збігів слів - добре відображає, чи вибирає модель правильну лексику;
- chrF++ оцінює морфологічну близькість через символні n-грами - важливо для українського, де одне слово може мати кілька відмінкових форм;
- TER показує мінімальний обсяг правок для отримання еталону - інтуїтивно зрозуміла метрика для оцінки «скільки ще треба виправити».

Для задачі нормалізації до близькоспорідненої літературної норми типові очікувані діапазони: chrF++ > 90 для суржику (мінімальні лексичні відмінності) та суттєво нижчі значення для діалектів із більшою фонетичною і морфологічною відстанню від норми, таких як закарпатський. Низький TER при невисокому BLEU може сигналізувати про правильні слова у неправильному порядку або часткові збіги.

## 2.7 Стратегії декодування

Задача декодування - знайти найбільш імовірну вихідну послідовність:

$$\hat{y} = \arg \max_y \log p_{\theta}(y|x) \quad (20)$$

**Жадібний пошук:** на кожному кроці вибирається лексема з найвищою ймовірністю:

$$\hat{y}_t = \arg \max_{y_t} p_{\theta}(y_t|x, \hat{y}_{<t}) \quad (21)$$

Найшвидший варіант, але може «зациклюватись» або пропустити глобально кращу послідовність.

**Пошук з променем** (beam search,  $B$  - розмір променя): підтримує  $B$  найкращих часткових гіпотез одночасно. Для umt5-base під час оцінювання  $B = 4$ . Краща якість за рахунок перебору, ціна - лінійне збільшення обчислень.

**Семплювання з температурою** (для MatayLM): лексема вибирається із розподілу, загостреного температурою  $\tau$ :

$$p'_{\theta}(y_t|x, \hat{y}_{<t}) \propto \exp\left(\frac{\log p_{\theta}(y_t|x, \hat{y}_{<t})}{\tau}\right) \quad (22)$$

При  $\tau = 0.1$  розподіл загострюється і наближається до жадібного пошуку, зберігаючи невелику варіативність. Додатково застосовується top-k фільтрування ( $k = 25$ ): решту лексем відкидають.

Відмінність стратегій: umt5-base генерує детермінований вихід (пошук з променем), MatayLM - з незначним різноманіттям через семплювання.

## 2.8 Балансування даних

### 2.8.1 Дисбаланс діалектів та принципи семплювання

Мультидіалектний корпус нерівномірний: суржик охоплює  $\approx 50$  тис. прикладів, тоді як гуцульський вручну - менше 10 тис. При навчанні на об'єднаному корпусі оптимізація зміщується у бік найбільшого підкорпусу. Для вирівнювання вводяться ваги семплювання:

$$P(\text{діалект} = i) \propto n_i^{\gamma} \quad (23)$$

де  $n_i$  - кількість прикладів діалекту  $i$ , а  $\gamma \in [0, 1]$  контролює ступінь вирівнювання ( $\gamma = 0$  - рівномірний,  $\gamma = 1$  - пропорційний до обсягу).

### **2.8.2 Формат інструкцій для мультидіалектності**

Для декодерної ВММ використовується єдиний формат із маркером діалекту в системній ролі:

- нормалізація гуцульського (маркер «HUTSUL»);
- нормалізація бойківського (маркер «BOIKO»);
- нормалізація закарпатського (маркер «TRANSCARPATHIAN»);
- нормалізація суржику (маркер «SURZHYK»).

Таким чином система розв’язує умовну задачу  $p_\theta(y|x, t)$ , де  $t$  - тип нормалізації, закодований у вхідному запиті.

## **2.9 Висновки до розділу**

У розділі наведено математичну постановку задачі нормалізації як умовної генерації  $p_\theta(y|x)$  та описано два реалізовані підходи. Кодер-декодерна модель umt5-base [3.] пройшла повне донавчання за 40 епох із кроком навчання  $5 \times 10^{-5}$ , згладжуванням міток  $\alpha = 0.1$  (формула 6) та раннім зупином. Декодерна ВММ MatayLM [11.] навчена через QLoRA ( $r = 16$ ,  $\alpha = 32$ , 4-bit NF4) за 3 епохи з кроком  $2 \times 10^{-4}$ . Описано математичні основи: механізм уваги трансформера (формула 7), цільові функції (формули 5–6), стратегії декодування (жадібний пошук та пошук з променем) і метрики BLEU [16.], chrF++ та TER (формули 16–19).

## 3 ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ

### 3.1 Технологічний стек

Систему реалізовано на Python 3.11 з фіксованими версіями залежностей у файлі `pyproject.toml`. Керування залежностями виконується через інструмент `uv`, а всі основні операції автоматизовано через `Makefile`: встановлення середовища (`install`), підготовка даних (`prepare-data`), генерація корпусів (`generate-*`), навчання моделей (`train-*`), оцінювання (`evaluate-*`), запуск застосунку (`demo`) та компіляція документації (`pdf`).

Перевага такого підходу - відтворюваність: будь-який дослідник може клонувати репозиторій і відтворити всі результати однією командою `make all`, не виконуючи ручних кроків. Зокрема, `make prepare-data` завантажує вихідні речення, запускає правилу та LLM-генерацію і формує фінальний навчальний корпус повністю автоматично. Цей підхід до автоматизації знижує ризик помилок відтворення, притаманних ручним інструкціям.

Основні бібліотеки, задіяні у системі, наведено в таблиці 5.

Бібліотека `Unsloth` є ключовою для прискорення QLoRA-навчання. Вона перезаписує критичні операції (функція уваги, обчислення `softmax` у `cross-entropy`) нативними ядрами `CUDA` через `Triton-JIT`-компіляцію, що дає зниження споживання відеопам'яті на 30-40% порівняно зі стандартною реалізацією `Transformers`. Додатково `Unsloth` підтримує асинхронне збереження контрольних точок у фоновому потоці, не призупиняючи навчання. Бібліотека `rumorphy2` забезпечує морфологічний аналіз для правилкової генерації суржику: без неї підтримка відмінкових форм заміників була б неможливою без великої вручну складеної таблиці.

Крім програмних залежностей, важливу роль відіграє технічне забезпечення. Різні етапи роботи системи ставлять різні вимоги до обчислювальних ресурсів (табл. 6).

Фактично у роботі використано сервер з `NVIDIA A100` (80 ГБ `VRAM`), 128 ГБ оперативної пам'яті та `NVMe`-диском обсягом 1 ТБ. Такі ресурси дозволили виконати всі етапи без обмежень за пам'яттю та сховищем. Найбільш ресурсомістким є етап генерації синтетики: модель `Mistral-Small-24B` потребує понад 16 ГБ `VRAM` навіть у 4-бітному режимі, а генерація 30 000 прикладів на діалект займає від 12 до 24 годин на `NVIDIA A100`.

Таблиця 5: Основні бібліотеки системи

| Бібліотека   | Версія        | Призначення                                                        |
|--------------|---------------|--------------------------------------------------------------------|
| PyTorch      | 2.6.0         | Основний фреймворк для навчання нейронних мереж                    |
| Transformers | $\geq 4.57.1$ | Трансформерні архітектури, токенизатори, конвеєри                  |
| PEFT         | $\geq 0.15.0$ | LoRA/QLoRA параметро-ефективне донавчання                          |
| TRL          | 0.18.2        | SFTTrainer для навчання з інструкціями                             |
| Unsloth      | 2026.1.4      | Прискорення QLoRA (2-3 рази): зли-та увага, оптимізований autograd |
| BitsAndBytes | $\geq 0.45.0$ | 4- та 8-бітне квантування вагових матриць                          |
| Accelerate   | $\geq 1.12.0$ | Розподілені обчислення та змішана точність                         |
| SacreBLEU    | $\geq 2.5.1$  | Обчислення BLEU, chrF++ та TER                                     |
| Datasets     | 3.4.1         | Завантаження та обробка наборів даних                              |
| NiceGUI      | $\geq 1.4.0$  | Вебзастосунок (Python-native UI над FastAPI)                       |
| PyAV (av)    | $\geq 14.0.0$ | Декодування аудіо WebM/OGG/WAV до 16 кГц                           |
| rumorphy2    | актуальна     | Морфологічний аналіз і відмінювання слів                           |
| Typeer       | $\geq 0.9.0$  | Інтерфейси командного рядка скриптів                               |

## 3.2 Конвеєр генерації корпусу

### 3.2.1 Правилова генерація суржику

Для суржику реалізовано клас `SurzhykErrorifier`, що перетворює нормативні українські речення у суржик без залучення зовнішніх ВММ. Такий підхід дозволив отримати додатково 20 911 прикладів за рахунок детермінованих правил, рівномірно покриваючи типові лексичні підстановки. Правильний підхід забезпечує стовідсоткову відтворюваність: однакове речення завжди дає однаковий суржиковий варіант.

Алгоритм складається з двох послідовних фаз.

**Фаза 1: фразова заміна.** Клас будує лексикон фраз: усі записи словника,

**Таблиця 6: Вимоги до технічного забезпечення за етапами**

| Компонент | Генерація синтетики | Навчання umt5 | QLoRA MamayLM | Застосунок |
|-----------|---------------------|---------------|---------------|------------|
| ЦП        | 8+ ядер             | 8+ ядер       | 8+ ядер       | 4+ ядра    |
| ОП        | 32 ГБ               | 16 ГБ         | 32 ГБ         | 8 ГБ       |
| ГП (VRAM) | 24 ГБ               | 8 ГБ          | 24 ГБ         | 4 ГБ       |
| Сховище   | 50 ГБ               | 10 ГБ         | 20 ГБ         | 5 ГБ       |

що складаються з кількох слів, упорядковуються за спаданням довжини у лексемах. Для кожного вхідного речення алгоритм застосовує ці фрази як регулярні вирази (з урахуванням меж слів \b) від найдовшої до найкоротшої, підставляючи відповідний суржиковий варіант при першому збігу. Пріоритет найдовшого збігу запобігає частковим замінам: наприклад, фраза «на самому ділі» замінюється цілком, а не лише слово «самом». Усі заміщені позиції у реченні позначаються як вже оброблені і не передаються до фази 2.

**Фаза 2: пословна морфологічна заміна.** Для кожного слова, ще не заміненого у фазі 1, виконується лематизація через `ru morphology2`. Якщо лема знайдена у лемному індексі словника, береться суржиковий відповідник. Далі `ru morphology2` відмінює його у потрібну граматичну форму, збігаючи граммами оригінального слова (рід, число, відмінок, особа, час). Це гарантує граматичну узгодженість результату: якщо вхід містить слово у родовому відмінку множини, суржиковий заміник також опиняється у тій самій формі. Якщо потрібна граматична форма відсутня у парадигмі замінника (наприклад, незмінювані слова), система повертає початкову форму без помилки.

Речення відкидається, якщо жодної заміни не відбулося. Відношення ефективності: з усього доступного корпусу нормативних речень правилний підхід дав позитивний результат (хоча б одну заміну) приблизно для 40% речень, що й склало 20 911 прикладів суржику.

### **3.2.2 LLM-генерація для діалектів**

Для генерації бойківського, закарпатського та гуцульського діалектів, а також додаткового суржику, реалізовано клас `DialectCorpusGenerator`. Він складається з двох взаємодіючих компонентів: `DialectPromptBuilder` та `LocalModelClient`.

**DialectPromptBuilder** відповідає за формування інструкційного запиту до моделі. Для кожного діалекту завантажується відповідний файл правил з те-

ки prompts/ (337-568 рядків). Щоб не включати до контексту весь словник (що може перевищувати ліміт лексем моделі), застосовується відбір найрелевантніших статей. Обчислюється TF-IDF косинусна схожість між вхідним реченням та кожним записом словника, і до запиту включаються лише  $k = 50$  найближчих статей. Таким чином кожне речення отримує сфокусовану лексичну підказку, адаптовану до його конкретної семантики: речення про транспорт отримує транспортну лексику, а речення про побут - побутову. Шаблон запиту зображено на рис. 3.1.

Відібрані словникові статті замінюють плейсхолдер {dictionary} у системному запиті. Запит користувача містить пакет з 5 вхідних речень, пронумерованих списком. Модель повертає відповідний список з 5 перекладів у форматі JSON-масиву рядків.

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."  
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."  
* Assistant: "...нормалізований текст..."
```

*Рис. 3.1: Шаблон інструкційного запиту для генерації діалектних пар.*

**LocalModelClient** виконує інференцію на локально розгорнутій моделі Mistral-Small-24B-Instruct-2501 з 4-бітним квантуванням NF4 у режимі подвійного квантування. Локальний запуск обрано замість хмарного API з двох причин: по-перше, це усуває залежність від зовнішніх сервісів і ліміти запитів; по-друге, забезпечує повний контроль над параметрами генерації (temperature=0.7, max\_tokens=2048). Пакетна обробка по 5 речень за запит дозволяє моделі розглядати речення у спільному контексті, що підвищує стилістичну узгодженість перекладів у межах одного пакету. При обробці ізольованих речень модель може обрати різний стиль для схожих фраз; спільний контекст 5 речень знижує таку варіативність.

Клас DialectCorpusGenerator реалізує чотирирівневий контроль якості:

- **Подвійний розбір відповіді:** спершу спроба декодувати JSON-масив рядків; якщо не вдається - текстовий запасний варіант (розбиття за нумерованими рядками).
- **Валідація:** відсіюються пари, де вихід коротший за третину входу (ознака незавершеності відповіді), або де присутні залишки JSON-синтаксису, або де кількість виходів не збігається з кількістю входів.

- **Дедуплікація:** хеш нормалізованого рядка (нижній регістр, без зайвих пробілів) зберігається у множині; дублікати не додаються до корпусу незалежно від сеансу генерації.
- **JSONL-журнал:** кожен прийнятий запис зберігається з полем `source`, що містить ідентифікатор діалекту та номер пакету. Це дозволяє відстежити походження будь-якого прикладу у корпусі.

Ліміт генерації встановлено на рівні 30 000 прикладів на діалект. Фактично отримано: 19 841 для гуцульського (ліміт не досягнуто через менший вихідний корпус речень), по 29 754 для бойківського та 29 612 для суржику. Повторні запуски при збоях безпечні завдяки дедуплікації: вже згенеровані приклади просто пропускаються.

### 3.2.3 Підготовка корпусу до навчання

Фінальний корпус формується скриптом `prepare_multidialect_data.py`. Він об'єднує підкорпуси всіх діалектів, перемішує їх і ділить на навчальну і валідаційну вибірки у пропорції 90/10. Тестова вибірка формується цілком окремо: вона містить виключно ручні анотації (по 50 прикладів на діалект, 200 разом) і жодним чином не перетинається з навчальними даними. Такий поділ гарантує чисте оцінювання: модель не могла «бачити» тестові речення у жодній формі під час навчання.

Розподіл підсумкового корпусу наведено в таблиці 7.

**Таблиця 7: Розподіл корпусу за різновидами**

| Різнovid      | Ручні        | LLM            | Прав.         | Навч.          | Валід.        | Тест       |
|---------------|--------------|----------------|---------------|----------------|---------------|------------|
| Гуцульський   | 9 853        | 19 841         | -             | 26 725         | 2 969         | 50         |
| Бойківський   | -            | 29 754         | -             | 26 778         | 2 976         | 50         |
| Закарпатський | -            | 29 605         | -             | 26 644         | 2 961         | 50         |
| Суржик        | -            | 29 612         | 20 911        | 45 471         | 5 052         | 50         |
| <b>Разом</b>  | <b>9 853</b> | <b>108 812</b> | <b>20 911</b> | <b>125 615</b> | <b>13 756</b> | <b>200</b> |

Підкорпус суржику є найбільшим завдяки двом незалежним підходам до генерації: LLM-синтетика і правилкова генерація не перетинаються за методом, тож їх об'єднання дає різноманітніший корпус. Підкорпус гуцульського діалекту вирізняється наявністю 9 853 ручних пар, що позитивно вплинуло на якість навчання. Підкорпуси бойківського та закарпатського діалектів сформовані виключно з LLM-синтетики через відсутність публічних паралельних корпусів



для цих різновидів.

### 3.3 Навчання моделей

#### 3.3.1 Донавчання *umt5-base*

Навчання кодер-декодерної моделі *umt5-base* [3.] реалізовано через клас `Seq2SeqTrainer` з бібліотеки `Transformers`. Процес складається з шести основних кроків.

**Крок 1: завантаження та підготовка даних.** CSV-файли з навчальною і валідаційною вибірками завантажуються в об'єкти `HuggingFace Dataset` з полями `source` (діалектний текст) та `target` (нормативна відповідь). Для мультидіалектної моделі поле `source` містить префікс, наприклад: «Переклади з гуцульської: *вхідний текст*». Префіксний підхід дозволяє єдиній моделі обслуговувати всі чотири різновиди без окремих ваг.

**Крок 2: токенізація.** Вхідні та цільові тексти кодуються через `AutoTokenizer` (`SentencePiece`, словник 250 112 лексем). Максимальна довжина входу і виходу обмежена 256 лексемами - цього достатньо для всіх речень у корпусі, оскільки середня довжина речення не перевищує 30 слів.

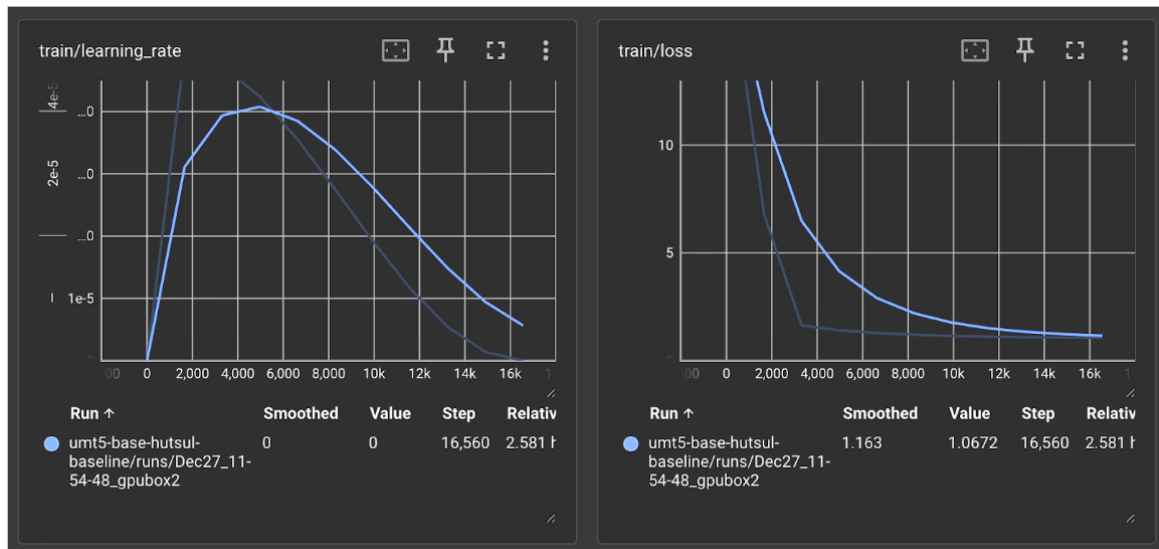
**Крок 3: динамічне доповнення.** Замість фіксованого доповнення до 256 лексем застосовується `DataCollatorForSeq2Seq` з параметром `pad_to_multiple_of=8`: кожен пакет доповнюється лише до кратної 8 довжини найдовшого речення у пакеті. Це мінімізує кількість паддінг-лексем і підвищує ефективність матричних операцій на CUDA, оскільки ядра тензорних обчислень оптимізовані для розмірів, кратних 8 або 16.

**Крок 4: навчання.** Використовується `Seq2SeqTrainer` з гіперпараметрами, описаними у підрозділі 2.5 (табл. 3). Функція втрат - перехресна ентропія зі згладжуванням міток  $\alpha = 0.1$  (формула 6), що запобігає надмірній впевненості моделі у окремих лексемах і покращує узагальнення.

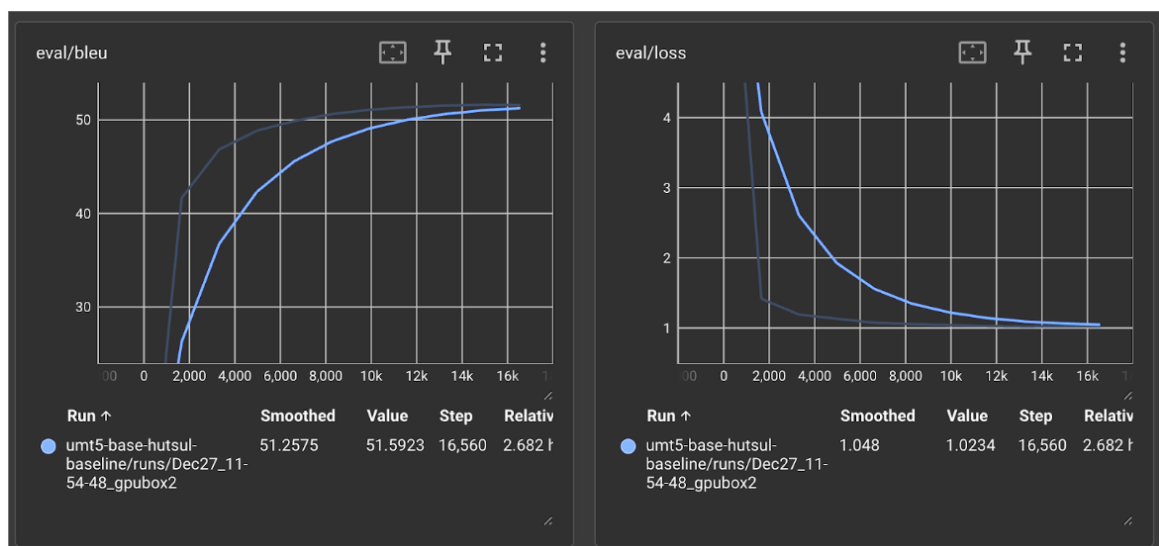
**Крок 5: моніторинг і рання зупинка.** Після кожної епохи обчислюється BLEU на валідаційній вибірці через функцію `compute_metrics`: декодуються передбачення (ігноруються спеціальні ID доповнення –100), обчислюється BLEU через `sacrebleu` з детоксифікацією. Якщо BLEU не покращується упродовж 5 епох поспіль, навчання зупиняється достроково. Зберігаються три останні контрольні точки; по завершенні автоматично завантажується найкраща за BLEU.

**Крок 6: збереження.** Фінальна модель і токенизатор зберігаються у каталог `models/umt5-base-multidialect/final_model`, звідки їх завантажує застосунок «Лексикон» у режимі реального часу.

Криву навчання базової моделі на гуцульському діалекті (перший, однодіалектний етап) показано на рис. 3.2, а динаміку BLEU на валідаційній вибірці - на рис. 3.3.



**Рис. 3.2:** Крива навчання *umt5-base* на гуцульському діалекті (функція втрат від кроку).



**Рис. 3.3:** Динаміка *BLEU* на валідаційній вибірці гуцульського діалекту за епохами.

### 3.3.2 Донавчання *MamayLM*

Навчання декодерної BMM *MamayLM* [11.] (Gemma-3-4B) реалізовано через `SFTTrainer` з бібліотеки `TRL` з прискоренням `Unsloth`. Ключова особливість

порівняно з кодер-декодерним підходом - необхідність явного форматування прикладів у чат-шаблон і маскування функції втрат.

**Крок 1: завантаження та квантування.** Базова модель завантажується у режимі 4-бітного квантування NF4 з подвійним квантуванням (BitsAndBytesConfig). Подвійне квантування означає, що коефіцієнти квантування самих ваг теж квантуються, що дає додаткову економію близько 0.4 біту на параметр. Якщо доступна бібліотека Unsloth - використовується її оптимізований завантажувач; інакше модель завантажується через стандартний Transformers разом з PEFT.

**Крок 2: форматування у чат-шаблон.** Кожен навчальний приклад форматується у два повідомлення: користувача («Переклади з {діалект}: {вхідний текст}») та асистента («{нормативний текст}»). Функція `apply_chat_template` перетворює ці пари у послідовність лексем відповідно до формату Gemma-3 (спеціальні маркери початку і кінця ходу). Префіксний метод обумовлення на діалект зберігається з `umt5`-підходу для сумісності.

**Крок 3: пакування.** `SFTTrainer` з параметром `packing=True` об'єднує короткі приклади у пакети до 512 лексем. Замість того щоб доповнювати кожен приклад до 512 лексем паддінгом (що давало б велику частку «порожніх» лексем), пакування заповнює кожен пакет реальним текстом майже повністю. Це суттєво підвищує завантаженість ГП і прискорює навчання: у практиці пакування дало приріст пропускної здатності близько 40% порівняно з непакованим режимом на тому самому обладнанні.

**Крок 4: маскування функції втрат.** Градієнт обчислюється лише за лексемами відповіді асистента. Лексеми системного запиту та запиту користувача замінюються на `-100` (ігнорований індекс у стандартній функції перехресної ентропії `PyTorch`), тому модель навчається генерувати нормативну відповідь, а не відтворювати вхідний запит.

**Крок 5: LoRA-адаптери.** QLoRA-адаптери ( $r = 16$ ,  $\alpha = 32$ , `dropout=0.0`) застосовуються до всіх проекційних матриць уваги та FFN-шарів: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`. Заморожені ваги базової моделі зберігаються у 4-бітному форматі і не займають оперативну пам'ять ГП в повній точності. Ефективна кількість навчальних параметрів - близько 13 мільйонів, що становить менш ніж 0.4% від загального обсягу моделі у 4 мільярди параметрів.

**Крок 6: контрольні точки та завершення.** Unsloth забезпечує асинхронне збереження контрольних точок у фоновому потоці без паузи у навчанні. Після завершення можливе злиття адаптерів у базову модель (`merge_and_unload`) для ефективного розгортання без накладних витрат PEFT під час інференції.

### 3.4 Оцінювання та аналіз результатів

#### 3.4.1 Методологія оцінювання

Тестова вибірка містить 200 прикладів: по 50 на кожен з чотирьох різновидів. Усі приклади зібрані вручну і жодним чином не перетиналися з навчальними даними. Ручна анотація тестових прикладів є принципово важливою: якби тестова вибірка містила LLM-синтетику, результати відображали б якість того самого генератора, яким навчалися моделі, а не реальну лінгвістичну точність.

Нуль-шотовими базовими лініями слугують три комерційні BMM, доступні через OpenRouter API: GPT-4o [16.], Gemini 2.0 Flash та Mistral Large. Їх запитували з тим самим системним запитом («Переклади з {діалект} до літературної української мови»), але без жодних прикладів у контексті - нуль-шотовий режим. Це дозволяє оцінити, наскільки спеціалізоване донавчання перевершує загальні BMM, що не мали доступу до наших навчальних даних.

Для оцінювання кодер-декодерних моделей реалізовано скрипт `evaluate_encoder_decoder.py`, для декодерних BMM - `evaluate_decoder_only.py`. Обидва зберігають результати у `results.json` з розбивкою показників за кожним з чотирьох різновидів. Обчислюються три метрики, описані у підрозділі 2.6: BLEU (формула 16), chrF++ та TER (формула 19).

#### 3.4.2 Результати кодер-декодерної моделі *umt5-base*

Зведені результати *umt5-base* по всіх різновидах наведено на рис. 3.4.

Найкращий результат - суржик ( $\text{chrF++} = 93.58$ ). Це пояснюється найбільшим навчальним підкорпусом (понад 50 000 пар двох типів генерації) та лексичною близькістю суржику до нормативної мови: більшість слів збігаються, відрізняються лише окремі лексеми. Гуцульський діалект демонструє  $\text{chrF++} = 86.90$ , що зумовлено найбільшою кількістю ручних анотацій (9 853 пари). Бойківський результат нижчий ( $\text{chrF++} = 54.78$ ): цей діалект має складніші синтаксичні і фонетичні відмінності від норми, а кодер-декодерна архітектура може недостатньо вловлювати їх при суто лексичних підстановках. Закарпатський

| UMT5-base (finetuned, encoder-decoder) |       |        |       |
|----------------------------------------|-------|--------|-------|
| Tasks                                  | bleu  | chrF++ | ter   |
| boiko_translation                      | 29.12 | 54.78  | 49.37 |
| hutsul_translation                     | 74.68 | 86.90  | 12.53 |
| surzhyk_translation                    | 89.42 | 93.58  | 5.98  |
| transcarpathian_translation            | 6.20  | 27.45  | 82.21 |

**Рис. 3.4: Результати umt5-base на тестовій вибірці за різновидами (BLEU, chrF++, TER).**

дає найслабший результат (chrF++ = 27.45): суттєва лінгвістична відстань від літературної норми і відсутність ручних прикладів у навчанні.

### 3.4.3 Результати декодерної BMM MamayLM

Результати MamayLM наведено на рис. 3.5.

| MamayLM (finetuned, decoder-only) |       |        |       |
|-----------------------------------|-------|--------|-------|
| Tasks                             | bleu  | chrF++ | ter   |
| boiko_translation                 | 51.81 | 66.82  | 35.26 |
| hutsul_translation                | 64.84 | 74.02  | 12.53 |
| surzhyk_translation               | 78.48 | 91.22  | 11.96 |
| transcarpathian_translation       | 3.82  | 20.93  | 89.69 |

**Рис. 3.5: Результати MamayLM на тестовій вибірці за різновидами (BLEU, chrF++, TER).**

MamayLM демонструє сильний результат на суржику (chrF++ = 91.22) і на бойківському (chrF++ = 66.82), де вона перевершує umt5-base. На гуцульському результат нижчий (74.02 проти 86.90 у umt5), що може пояснюватися меншою кількістю навчальних кроків (3 епохи проти 40) і природою декодерних BMM: такі моделі краще виявляють семантичні паттерни, але гірше «запам'ятовують» точні лексичні відповідності, важливі для символічних метрик. Закарпатський залишається найскладнішим і для MamayLM (chrF++ = 20.93).

### 3.4.4 Порівняння з нуль-шотовими базовими лініями

Результати нуль-шотових базових ліній наведено на рисунках 3.6-3.9.

Зведену таблицю chrF++ для всіх моделей і різновидів наведено в таблиці 8.

| Tasks                       | Filter | n-shot | Metric      | Value   | Stderr | Stderr_CLT |
|-----------------------------|--------|--------|-------------|---------|--------|------------|
| boiko_translation           | none   | 0      | chrF_score↑ | 0.7395± | N/A    | 0.0199     |
| hutsul_translation          | none   | 0      | chrF_score↑ | 0.6489± | N/A    | 0.0255     |
| surzhyk_translation         | none   | 0      | chrF_score↑ | 0.8897± | N/A    | 0.0176     |
| transcarpathian_translation | none   | 0      | chrF_score↑ | 0.6539± | N/A    | 0.0218     |

*Рис. 3.6: Результати нуль-шотового GPT-4o за різновидами.*

| Tasks                       | Filter | n-shot | Metric      | Value   | Stderr | Stderr_CLT |
|-----------------------------|--------|--------|-------------|---------|--------|------------|
| boiko_translation           | none   | 0      | chrF_score↑ | 0.7457± | N/A    | 0.0199     |
| hutsul_translation          | none   | 0      | chrF_score↑ | 0.6534± | N/A    | 0.0245     |
| surzhyk_translation         | none   | 0      | chrF_score↑ | 0.8667± | N/A    | 0.0193     |
| transcarpathian_translation | none   | 0      | chrF_score↑ | 0.6816± | N/A    | 0.0210     |

*Рис. 3.7: Результати нуль-шотового Gemini 2.0 Flash за різновидами.*

### 3.4.5 Аналіз результатів

Аналіз показників дозволяє зробити кілька спостережень.

**Суржик.** Обидві донавчені моделі перевершують GPT-4o (93.58 і 91.22 проти 88.97 за chrF++). Основна причина - великий корпус (понад 50 000 пар) та лексична близькість суржику до норми: більшість слів ідентична, змінюється лише 20-30% лексики. Такий порівняно «легкий» режим дозволяє моделям зі спеціалізованим навчанням надійно запам'ятовувати конкретні підстановки, тоді як нуль-шотові BMM роблять узагальнені припущення без знання конкретних пар.

**Гуцульський.** umt5-base (86.90) і MamayLM (74.02) суттєво перевищують GPT-4o (64.89). Вирішальним фактором став найбільший ручний підкорпус (9 853 пари): моделі вивчили специфічну фонетику і лексику гуцульського, яку GPT-4o знає лише з передтренувального корпусу. Різниця між umt5 і MamayLM на гуцульському (12.88 за chrF++) пояснюється більшою кількістю навчальних кроків umt5 (40 епох) і повним донавчанням без заморожування ваг.

**Бойківський.** MamayLM (66.82) перевершує umt5-base (54.78). Бойківський діалект має більше синтаксичних і словотвірних відмінностей від норми, де сильніший мовний контекст декодерної BMM дає перевагу. umt5-base - строга seq2seq-модель, що генерує лексему за лексемою без авторегресивного контексту відповіді; MamayLM, навпаки, кожену нову лексему генерує з урахуванням уже виданих, що підвищує граматичну зв'язність виводу.

**Закарпатський.** Найнижчі результати серед усіх різновидів (umt5: 27.45, MamayLM: 20.93). При цьому GPT-4o показує 65.39, що виглядає парадоксаль-

| Tasks                       | Filter | n-shot | Metric       | Value   | Stderr | Stderr_CLT |
|-----------------------------|--------|--------|--------------|---------|--------|------------|
| boiko_translation           | none   | 0      | chr_f_score↑ | 0.7091± | N/A    | 0.0225     |
| hutsul_translation          | none   | 0      | chr_f_score↑ | 0.6318± | N/A    | 0.0233     |
| surzhyk_translation         | none   | 0      | chr_f_score↑ | 0.8579± | N/A    | 0.0209     |
| transcarpathian_translation | none   | 0      | chr_f_score↑ | 0.6365± | N/A    | 0.0250     |

*Рис. 3.8: Результати нуль-шотового Mistral Large за різновидами.*

| MamayLM (baseline, before finetuning) |       |         |       |  |  |  |
|---------------------------------------|-------|---------|-------|--|--|--|
| Tasks                                 | bleu  | chr_f++ | ter   |  |  |  |
| boiko_translation                     | 21.77 | 58.73   | 56.42 |  |  |  |
| hutsul_translation                    | 5.06  | 16.73   | 50.13 |  |  |  |
| surzhyk_translation                   | 49.07 | 65.58   | 35.89 |  |  |  |
| transcarpathian_translation           | 10.43 | 27.90   | 82.21 |  |  |  |

*Рис. 3.9: Порівняння MamayLM (донавченого) з нуль-шотовими базовими лініями.*

но для моделі без донавчання. Причина криється у якості навчальних даних: корпус закарпатського сформований виключно з LLM-синтетики (Mistral Small 24B). Синтетичні дані можуть містити систематичні спрощення, що не відображають реальну лінгвістичну відстань закарпатського від норми. GPT-4o спирається на знання з власного передтреновального корпусу і більш адекватно обробляє незнайомий діалект без таких артефактів. Ця ситуація вказує на критичну потребу в ручній розмітці для закарпатського.

**Загальний висновок:** спеціалізоване донавчання виправдане і перевіряє нуль-шотові ВММ для тих різновидів, де є якісні навчальні дані (суржик, гуцульський). Для рідкісних діалектів без ручної розмітки (закарпатський) якість LLM-синтетики є критичним вузьким місцем.

### 3.5 Вебзастосунок «Лексикон»

#### 3.5.1 Архітектура застосунку

Застосунок «Лексикон» реалізовано на базі фреймворку NiceGUI версії 1.4 і вище. NiceGUI будує Python-native UI над FastAPI і Starlette: розробник описує інтерфейс через Python-об'єкти (`ui.button`, `ui.textarea` тощо), а фреймворк генерує відповідний HTML/JavaScript і обслуговує його через вбудований ASGI-сервер. Перевага такого підходу - відсутність необхідності писати JavaScript вручну: весь код інтерфейсу, моделі та обробки запитів знаходиться в одному

**Таблиця 8: Порівняння моделей за метрикою chrF++ на тестовій вибірці**

| Модель           | Суржик       | Гуцуль-ський | Бойків-ський | Закар-патський |
|------------------|--------------|--------------|--------------|----------------|
| umt5-base (наш)  | <b>93.58</b> | <b>86.90</b> | 54.78        | <b>27.45</b>   |
| MamayLM (наш)    | 91.22        | 74.02        | <b>66.82</b> | 20.93          |
| GPT-4o           | 88.97        | 64.89        | -            | 65.39          |
| Gemini 2.0 Flash | -            | -            | -            | -              |
| Mistral Large    | -            | -            | -            | -              |

Python-середовищі.

Застосунок запускається командою `make demo` (або безпосередньо `uv run python app.py`) і доступний за адресою `http://0.0.0.0:7870`. При старті завантажуються обидва модулі: нормалізатор і модуль АРМ. Якщо ГП недоступний, нормалізатор автоматично перемикається на ЦП-режим (`float32` замість `float16`), що дозволяє запустити застосунок навіть без відеокарти з певним зниженням швидкості.

### **3.5.2 Модуль нормалізації**

Клас `DialectTranslator` завантажує збережену мультидіалектну модель `umt5-base` з каталогу `models/umt5-base-multidialect/final_model` у режимі `float16` на ГП. Обумовлення на конкретний діалект реалізується через префікс запиту:

- «surzhyk» → «Переклади з суржику: {текст}»;
- «hutsul» → «Переклади з гуцульської: {текст}»;
- «boiko» → «Переклади з бойківської: {текст}»;
- «transcarpathian» → «Переклади з закарпатської: {текст}».

Вхідний текст токенізується і передається на генерацію через `model.generate()` з такими параметрами: `num_beams=5` (пошук з 5 променями), `max_length=128`, `repetition_penalty=1.2`, `no_repeat_ngram_size=3`. Параметр `repetition_penalty=1.2` знижує ймовірність повторення слів у виводі, а `no_repeat_ngram_size=3` забороняє повтор будь-якого тримграму - разом це усуває артефакти зациклення, характерні для `seq2seq`-моделей на коротких або незнайомих входах.

Користувач може динамічно регулювати `num_beams` (від 3 до 10) та `repetition_penalty` (від 1.0 до 2.0) через повзунки в інтерфейсі без перезаван-



таження моделі. Зміна `num_beams` впливає на якість і швидкість: більше променів дає точніший результат, але збільшує час відповіді.

### ***3.5.3 Модуль автоматичного розпізнавання мовлення***

Модуль АРМ використовує модель `speechbrain/m-ctc-t-large` (МСТСТForСТС через Hugging Face Transformers) - багатомовну СТС-модель, попередньо навчену на 7 мовах без додаткового донавчання на діалектних даних. Модель обрана через підтримку кириличного алфавіту і відносно невеликий розмір (близько 450 МБ) для моделі такого класу.

Обробка аудіо реалізована через бібліотеку PyAV: браузер записує мовлення через MediaRecorder API і надсилає файл у форматі WebM або OGG на ендпоінт FastAPI `POST /api/transcribe`. На сервері PyAV декодує аудіопотік, конвертує у моно `float32` і ресемплює до 16 кГц - стандартна частота дискретизації для більшості АРМ-моделей. Отриманий масив передається у модель, результат декодується через `AutoProcessor` і повертається у відповіді як JSON: `{"text": "..."}.`

Перевага використання PyAV замість зовнішнього `ffmpeg`: бібліотека встановлюється через `pip` і містить всі необхідні кодеки статично, що спрощує розгортання на нових машинах без системних залежностей.

### ***3.5.4 Інтерфейс користувача***

Інтерфейс організований у три колонки на десктопі: ліва - панель вибору діалекту з прикладами, центральна - поле введення тексту, права - поле результату. На мобільних пристроях (ширина менш ніж 768 пікселів) колонки перевлаштовуються у вертикальний стек через CSS breakpoint.

Функціонал застосунку:

- чотири кнопки вибору діалекту з коротким описом і географічним регіоном;
- поле введення тексту з кнопкою мікрофону для голосового введення через АРМ - натискання починає запис, повторне натискання зупиняє і автоматично транскрибує мовлення у поле введення;
- кнопка «Нормалізувати» запускає обробку і виводить результат у правому полі;
- повзунки для тонкого налаштування параметрів генерації (`num_beams`, `repetition_penalty`);

- чотири приклади вхідних речень (по одному на кожен різновид) для швидкого тестування без введення тексту вручну.

Весь інтерфейс описано приблизно у 250 рядках Python-коду у файлі `app.py`, що є наслідком декларативного підходу NiceGUI.

### 3.6 Висновки до розділу

У розділі описано програмне та технічне забезпечення системи «Лексикон». Технологічний стек базується на Python 3.11 з бібліотеками PyTorch 2.6.0, Transformers, PEFT, TRL та Unsloth; навчання відбувалося на NVIDIA A100 (80 ГБ VRAM). Конвеєр генерації корпусу реалізує два підходи: правилний через клас `SurzhykErrorifier` (20 911 прикладів суржику з `rumorphy2` і словника) та LLM-генерація через клас `DialectCorpusGenerator` (108 812 прикладів через `Mistral-Small-24B-Instruct-2501`). Загалом корпус налічує 125 615 навчальних пар.

Навчання `umt5-base` виконано через `Seq2SeqTrainer` за 40 епох з раннім зупином (терпіння 5 епох), навчання `MamayLM` - через `SFTTrainer` з `QLoRA` та пакуванням за 3 епохи. Оцінювання на 200 ручних прикладах показало, що донавчені моделі перевершують нуль-шотові BMM на суржику (`chrF++` = 93.58 для `umt5-base` проти 88.97 у `GPT-4o`) та гуцульському (86.90 проти 64.89). Закарпатський діалект залишається найскладнішим через відсутність ручної розмітки у навчальних даних. Вебзастосунок реалізовано на NiceGUI і FastAPI з нормалізатором на основі `umt5-base` і APM-модулем `speechbrain/m-ctc-t-large`.

## ЗАГАЛЬНІ ВИСНОВКИ

У роботі розроблено систему автоматичної нормалізації суржику та діалектних форм до літературної української мови.

Основні результати роботи:

1. Проаналізовано предметну область та виконано огляд сучасних підходів до нормалізації діалектів і гібридних мовних форм у задачах NLP.
2. Проаналізовано доступні джерела даних для задачі “діалект/суржик → літературна українська” та обґрунтовано спосіб формування навчального корпусу.
3. Розроблено та експериментально оцінено baseline-рішення на прикладі гуцульського діалекту.
4. На основі отриманих результатів масштабовано підхід на кілька діалектів та суржик.
5. Виконано інструкційне донавчання сучасної україномовної LLM (LaraLLM) для задачі нормалізації.
6. Реалізовано прикладну систему з інтерфейсом користувача для демонстрації роботи.

Система придатна для приведення коротких текстів до літературної норми і може слугувати окремим інструментом або компонентом більших україномовних NLP-пайплайнів.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. L. H. Baniata, S. Park, i S.-B. Park, “A Neural Machine Translation Model for Arabic Dialects That Utilizes Multitask Learning (MTL)”, *Computational Intelligence and Neuroscience*, 2018, с. 1-10. doi: 10.1155/2018/7534712.
2. M. Bondarenko, A. Yushko, A. Shportko, i A. Fedorych, “Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction”, *Proceedings of the Third Ukrainian Natural Language Processing Workshop (UNLP)*, 2024.
3. H. W. Chung et al., “UniMax: Fairer and more Effective Language Sampling for Large-Scale Multilingual Pretraining”, КВІТЕНЬ 2023, arXiv:2304.09151. doi: 10.48550/arXiv.2304.09151.
4. R. Fedchuk i V. Vysotska, “Mathematical Model of a Decision Support System for Identification and Correction of Errors in Ukrainian Texts Based on Machine Learning”, *CEUR Workshop Proceedings*, 2021.
5. M. Haltiuk, “On the Path to Make Ukrainian a High-Resource Language”, *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, 2023.
6. K. Kent, “Language Contact: Morphosyntactic Analysis of Surzhyk Spoken in Central Ukraine”, *Journal of Language Contact*, 2010.
7. A. Kiulian et al., “From Bytes to Borsch: Fine-Tuning Gemma and Mistral for the Ukrainian Language Representation”, КВІТЕНЬ 2024, arXiv:2404.09138. doi: 10.48550/arXiv.2404.09138.
8. R. Kovalchuk, M. Romanyshyn, i P. Ivaniuk, “Introducing OmniGEC: A Silver Multilingual Dataset for Grammatical Error Correction”, *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, 2023.
9. R. Kyslyi, Y. Maksymiuk, i I. Pysmennyi, “Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation”, ЧЕРВЕНЬ 2025, arXiv:2506.07617. doi: 10.48550/arXiv.2506.07617.
10. Lapa LLM.  
Доступно у: <https://huggingface.co/lapa-llm>
11. MamayLM.

Доступно у: [https://huggingface.co/blog/INSAIT-Institute/mamay\\_lm](https://huggingface.co/blog/INSAIT-Institute/mamay_lm)

12. A. Saini, A. Chernodub, V. Raheja, i V. Kulkarni, “Spivavtor: An Instruction Tuned Ukrainian Text Editing Model”, Квітень 2024, arXiv:2404.18880. doi: 10.48550/arXiv.2404.18880.
13. N. Sira, G. M. Di Nunzio, i V. Nosilia, “Towards an Automatic Recognition of Mixed Languages in R: The Case of Ukrainian-Russian Hybrid Language Surzhyk”, *CLiC-it 2021*, 2021.
14. O. Syvokon, O. Nahorna, P. Kuchmiichuk, i N. Osidach, “UA-GEC: Grammatical Error Correction and Fluency Corpus for the Ukrainian Language”, *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, 2023, с. 96-102. doi: 10.18653/v1/2023.unlp-1.12.
15. L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models”, *Transactions of the Association for Computational Linguistics*, 2022, arXiv:2105.13626. doi: 10.48550/arXiv.2105.13626.
16. H. Zhao et al., “From Handcrafted Features to LLMs: A Brief Survey for Machine Translation Quality Estimation”, *IJCNN 2024*, Чер 2024, с. 1-10. doi: 10.1109/IJCNN60899.2024.10650457.

## **Додаток А**

## **Додаток Б**